

ECOM EXPERTS

Orbit universe

Manual de usuario, técnico y de desarrollo

orbit universe · 08/09/2026

Contenido

Arrancar

Lo que estás viendo

Escenarios (491)

Los fundacionales

¿Qué pasa si...?

Mover un planeta de órbita

Viajar en el tiempo

Navegación

Ratón

Táctil (móvil y tableta)

Teclado

Panel

Lo que es real y lo que está exagerado

Rendimiento

Aula: usarlo en clase

En el móvil

Subirlo a un servidor

Documentación

1. Árbol

2. Unidades

3. El modelo de curvatura (core/warp.js)

Lo que se ve no es el desplazamiento, es la deformación

Modo 2D: paraboloide de Flamm

Ondas gravitatorias

4. Agujero negro por trazado de geodésicas (view/blackhole.js)

5. Física (core/physics.js)

5 bis. Geometría del campo (core/orbital.js)

5 ter. Qué tamaño se dibuja cada cuerpo

La excepción inevitable: un satélite y su planeta

Y lo mismo con las cáscaras y con el encuadre

Cómo se comprueba

5 quater. El editor de órbitas

Anfitrión ≠ quien más tira

5 quinquies. Muestreo de las trazas

5 sexies. El buscador de escenarios

5 septies. Familias con umbral, y cómo se comprueban

5 octies. Tamaños y encuadre: todo en píxeles de pantalla

Las cuatro causas, medidas

La regla, ahora

El encuadre

Lo que sigue sin ser perfecto, y por qué

Cómo se comprueba

5 nonies. El encuadre se MIDE, no se calcula

5 decies. El pozo se mide contra lo que orbita

5 undecies. Un agujero negro no ocupa lo que su horizonte

5 duodecies. Viaje en el tiempo

Las fechas

5 terdecies. El nacimiento de un sistema

5 quaterdecies. Polvo, círculos de distancia y la ficha del escenario

kind: 'polvo'

Círculos de distancia

La ficha del escenario y sus tiempos

5 quindecies. El repaso a fondo: qué se probó y qué salió

Los botones «y si...» (dev/whatif.py)

Las capas (dev/capas.py)

La física (dev/fisica.py)

El catálogo

Lo único que salió roto de verdad

6. Rendimiento

7. Detalles que costaron y conviene no repetir

8. Prueba de humo

9. Subida a un subdominio

9.1 Qué subdominio

9.2 Qué se sube

9.3 Requisitos del servidor

9.4 Comprobación después de subir

10. El aula (js/data/lecciones.js + js/ui/aula.js)

11. Móvil: qué se reorganizó y por qué

12. SEO: qué se añadió y para qué sirve cada cosa

13. El impacto gigante: cuatro fallos que no daban ningún error

13.1 La eyecta nacía en una posición caducada

13.2 Los escombros iban sub veces más despacio que los cuerpos

13.3 El anfitrión del disco era el Sol

13.4 Y cuando por fin nacía, la Tierra se la comía

13.5 El resultado, contrastado

13.6 La lección

14. Viaje en el tiempo: no perder de vista lo que se está mirando

15. El paso de integración es FIJO, y por qué eso importa tanto

La corrección

La comprobación

El avance rápido no toca esto

16. Transferencia de masa por lóbulo de Roche

El modelo

Las consecuencias, que es lo que se ve

Medido

Cinco fallos que costaron, y ninguno daba error

Y la lección de todo esto

1. El problema de las láminas 2D

2. Unidades

3. La curvatura que se dibuja

3.1 Modo 3D — contracción radial

3.2 Modo 2D — paraboloides de Flamm (exacto)

3.3 Lo que decide la diferencia entre Sol, enana blanca, estrella de neutrones y agujero negro

3.4 La exageración

4. Órbitas — sin exagerar nada

4 bis. Precesión relativista del perihelio (1PN)

4 ter. Geometría del campo

5. Luz

Campo fuerte: el agujero negro

6. Dilatación temporal

7. Colisiones: nube de partículas, no una fusión instantánea

8. Ondas gravitatorias

9. Materia oscura

10. Relaciones masa-radio del laboratorio

11. Resumen honesto

Simulador de curvatura gravitatoria en 3D, con 491 escenarios de física real y un aula con lecciones guiadas.

Ver [README.md](#) (uso, aula y móvil), [DOC-TECNICO.md](#) (arquitectura, subida a un subdominio y SEO) y [DOC-FISICA.md](#) (modelo físico y aproximaciones).

Arranque en local: `node server.js 8877` -> <http://localhost:8877/> Subdominio previsto:
<https://orbitas.gmartos.es/>

Orbit Universe — guía de uso

Simulador de curvatura gravitatoria **en volumen**. La idea de partida: todas las ilustraciones de "la gravedad curva el espacio" usan una lámina 2D hundida. Aquí esa misma física se representa en 3D, y se puede **volar por dentro**.

Arrancar

```
cd "z:\modulos ps mios finalizados y ok\modulos IA\orbit universe"
node server.js 8877
```

→ <http://localhost:8877/>

Sin dependencias externas, sin `npm install`, sin build. Three.js está incluido en `vendor/`. Para subirlo a un subdominio basta con copiar la carpeta entera a un hosting estático: no hace falta Node en producción.

Lo que estás viendo

Capa	Qué es
Retícula volumétrica 3D	Una malla de coordenadas en el espacio. Se comprime hacia las masas: es el equivalente en 3D del hundimiento de la lámina. Solo brilla donde de verdad se deforma; el espacio plano se apaga solo.
Cáscaras equipotenciales	Esferas concéntricas alrededor de cada masa. Con dos cuerpos cerca se abollan y aparecen los lóbulos de Roche.
Lámina 2D	El dibujo clásico, pero bien hecho: es el paraboloide de Flamm , la incrustación exacta de una rebanada ecuatorial de Schwarzschild.
Geodésicas de luz	Rayos de luz reales, integrados con la desviación de la relatividad general (el doble que Newton).
Trazas orbitales	El recorrido de cada cuerpo. Se deforman con la geometría, como todo lo demás.
Agujeros negros	No son un círculo negro: cada píxel traza su geodésica nula hacia atrás. De ahí salen el anillo de fotones, el arco del disco por encima del agujero y el arrastre de las estrellas del fondo.
Geometría del campo	Puntos de Lagrange L1–L5 , esfera de Hill , límite de Roche y zona habitable . Nada está puesto a mano: se resuelven las ecuaciones con las masas y posiciones de ese instante, así que si tocas una masa se mueven solos. L4 y L5 se pintan en verde cuando son estables ($M_1/M_2 > 24,96$) y en ámbar cuando no.

Colores de la malla: azul = espacio plano · violeta = curvatura moderada · naranja/amarillo = fondo del pozo.

Escenarios (491)

Son demasiados para un desplegable a secas, así que arriba del todo hay un **buscador**. Escribe dos palabras y se queda con lo que las lleve las dos, mirando el título, la explicación y el tipo, sin importar acentos ni mayúsculas: *theia*, *marce cerca*, *agujero negro*, *marea*, *kepler*. Al lado hay un filtro por tipo, y cada cabecera del desplegable dice cuántos quedan en su grupo.

Los ocho grupos:

Grupo	Cuántos	Qué hay
Fundamentos	47	Los casos base y las familias con umbral: separación de Hill, Kozai-Lidov, troyanos, resonancias, velocidad de escape
Sistema solar	25	El sistema real, la Luna a siete distancias, satélites cruzando el límite de Roche
Sistemas planetarios reales	84	Más de cincuenta sistemas medidos de verdad, de TRAPPIST-1 a HR 8799, y la zona habitable según el tipo de estrella
Objetos compactos	45	Enanas blancas, púlsares, agujeros negros de nueve masas distintas, Sagitario A, <i>M87</i>
Choques y cataclismos	60	La matriz completa de choques entre los siete tipos de objeto, más los cataclismos reales
Gran escala	19	Cúmulos, discos, cinturones, galaxias con y sin materia oscura
Y si... (mover planetas)	164	Cada planeta llevado a la órbita de cada otro, con la masa cambiada, inclinado, retrógrado o borrado
Teóricos, imposibles e inventados	23	Mundos anillo, rosetas de Klemperer, agujeros de gusano, el Sol vaciado por dentro
Nacimiento de un sistema	13	De una nube de gas al sistema solar de hoy, por etapas, y nubes de otras masas

Casi todo sale de tablas con datos reales y de generadores, no de escenarios escritos a mano uno a uno. Eso tiene una consecuencia que importa: los números que aparecen en la explicación (periodo, insolación, radio de Schwarzschild, distancia de Hill) se **calculan**, así que no pueden contradecir a lo que hace la simulación.

Muchas familias están montadas alrededor de un **umbral físico exacto**, con casos a los dos lados: dos planetas son estables por encima de 3,46 radios de Hill mutuos y caóticos por debajo; los puntos L4 y L5 solo sujetan si la relación de masas pasa de 24,96; el mecanismo de Kozai-Lidov no hace absolutamente

nada por debajo de 39,2 grados. Nadie ha programado la diferencia: sale del cálculo. Recorre la familia entera y la verás aparecer.

Los fundacionales

Fundamentos — Sol + Tierra · Comparativa de pozos (Sol / enana blanca / estrella de neutrones / agujero negro con la misma regla) · Laboratorio de masa · Tipos de estrella O B A F G K M + enana marrón · Escala de las gigantes (Sol, Aldebarán, Betelgeuse, UY Scuti) · Precesión del perihelio de Mercurio · **Materia exótica de masa negativa**: su pozo no es un pozo, es una colina, y repele en vez de atraer.

Sistema solar — Interior + Júpiter con las galileanas · Sistema completo con los ocho planetas y Plutón · Puntos de Lagrange con los troyanos de Júpiter · Planeta errante · Resonancia de Laplace (Ío 4 : Europa 2 : Ganímedes 1) · **Lluvia de errantes** (cinco mundos expulsados: unos pasan de largo y otros quedan capturados por Júpiter, comprobado midiendo la energía, no a ojo) · **Planeta Nueve** (hipotético, con los transneptunianos que lo delatan) · **Planeta doble Plutón-Caronte**.

Sistemas planetarios reales — TRAPPIST-1 (7 planetas) · Próxima Centauri (b, c, d) · Alfa Centauri A y B · Kepler-16b, el planeta con dos soles · PSR B1257+12, los primeros exoplanetas · 55 Cancri (5 planetas).

Objetos compactos — Agujero negro + estrella · Binario de agujeros negros · Sagitario A y *las estrellas S* · TON 618 (66.000 millones de masas solares, sin exagerar la curvatura) · *Púlsar binario de Hulse-Taylor* · Kilonova GW170817 · *Fusión GW150914* · *Lóbulos de Roche (transferencia de masa)* · *Anillo de Einstein**.

Choques y cataclismos — Colisión planetaria · Impacto gigante Tierra-Theia · **Impacto gigante a escala real** (tamaños sin exagerar: se ve el brazo espiral de escombros y la Luna formándose sola del disco) · Choque de estrellas · Problema de los tres cuerpos · **Tres soles y sus planetas** (el caso caótico, con un mundo habitable que acaba desestabilizado) · Límite de Roche · Si desapareciera el Sol / Júpiter / la Tierra.

Gran escala — Materia oscura (halo) · Cúmulo estelar de 40 estrellas · Vía Láctea y Andrómeda.

¿Qué pasa si...?

Hay tres escenarios dedicados con el suceso **programado en el tiempo**, para que te dé tiempo a colocarte y ver el momento exacto:

Escenario	Qué pasa
Si desapareciera el Sol	A los 200 días el Sol se esfuma y cada planeta sale disparado en línea recta , tangente a donde estaba. La cámara viaja con la Tierra.
Si desapareciera Júpiter	A los 300 días Júpiter se va, los puntos de Lagrange dejan de existir y los troyanos se dispersan.
Si desapareciera la Tierra	A los 120 días la Tierra se va y la Luna se queda sola, convertida en un planeta más.

El HUD dice en todo momento si el objetivo sigue **ligado** (y con qué semieje) o si ya está **escapando**, y de quién. Con **G** la cámara se pega al objetivo y viaja con él: sin eso, en cuanto quitas el Sol los planetas se te

van de la pantalla en dos segundos.

Además, cada escenario trae sus propios botones: *quitar la Luna*, *quitar Júpiter*, *Júpiter $\times 10$ de masa*, *parar la Tierra en seco*, *apagar el Sol...* Y en la lista de cuerpos, cada uno tiene los suyos:

Botón	Qué hace
	enfocar
$\times 2$ / $\div 2$	duplicar o reducir su masa (y con ella su pozo y su rs)
	pararlo en seco: a partir de ahí solo lo mueve la gravedad
	eliminarlo del sistema

Al eliminar un cuerpo **su órbita anterior se queda dibujada en gris** y las de los demás se repintan desde cero, así que la comparación entre el antes y el después es directa. Quita la Luna y mira cómo se bambolea la Tierra; multiplica Júpiter por diez y mira cómo se le caen encima las galileanas.

Mover un planeta de órbita

Haz clic en cualquier cuerpo y en su ficha, abajo, hay un **editor de órbita**: cambias a qué distancia está de su anfitrión, con qué excentricidad, con qué inclinación y en qué sentido gira, y le das a *Mover a esa órbita*. El cuerpo se recoloca conservando la dirección en la que estaba y recibe la velocidad de la órbita nueva. A partir de ahí **decide la simulación**: puede quedarse tan tranquilo, entrar en resonancia con un vecino, cruzarse con él o salir despedido del sistema. Nada de eso está guionizado.

Los atajos se calculan sobre la escena que tienes delante, así que cambian con el escenario: *-20 %*, *$\times 2$* , *zona habitable*, *entre Venus y la Tierra*, *más allá de Neptuno*. Y mientras mueves el campo, debajo te va diciendo cuánto duraría el año y si esa distancia cae dentro o fuera de la zona habitable.

Funciona con cualquier cosa, no solo con planetas: prueba a llevarte la Luna al doble de distancia y comprobar que el mes pasa de 27 a 77 días. Para un satélite el campo cambia de unidad solo, a millones de kilómetros, porque "0,0025581 UA" no le dice nada a nadie.

Viajar en el tiempo

El mando de velocidad llega a 20.000 días por segundo, o sea 55 años por segundo: para ver un millón de años harían falta cinco horas mirando la pantalla. En el panel hay una sección **Viaje en el tiempo** con saltos de 1 año, 100, 10.000, un millón y diez millones, hacia delante y hacia atrás.

Los saltos cortos se **integran de verdad**, paso a paso, con el mismo integrador que el bucle normal: el resultado es idéntico a haberlo visto pasar, con todas las perturbaciones. Los saltos largos no caben (un millón de años son mil cien millones de pasos) y ahí se hace lo mismo que hacen las efemérides profesionales: **propagación kepleriana**, cada cuerpo por su órbita alrededor de su anfitrión. Es exacto para dos cuerpos y muy bueno en un sistema jerárquico, pero **no recoge las perturbaciones entre planetas**, y te lo dice en pantalla cuando lo hace.

Es reversible: ve diez millones de años hacia delante y otros diez hacia atrás y la Tierra vuelve a 0,9833 UA y la Luna a 0,3647 UL, exactamente donde estaban.

La fecha del HUD se adapta a la escala: fecha de calendario cerca del presente, «año 12.000» hasta el millón, «dentro de 4,38 M de años» más allá, y «hace 10,089 mil M de años» pasados los mil millones.

Navegación

Hay **dos modos**, y se cambian con la tecla **O** o con el mando de la esquina:

- **Órbita** (por defecto): arrastras y **giras alrededor del objetivo**. Es lo que todo el mundo espera de un visor 3D.
- **Nave**: arrastras y giras la cabeza; te desplazas con WASD. **Alt** + arrastrar orbita alrededor del objetivo sin salir de este modo.

En la esquina hay un **mando en pantalla** con todo: cruceta para rotar, zoom, subir y bajar, alabeo, encuadrar el objetivo, encuadrar todo, vista inicial, seguir y giro automático. Así no hay que adivinar ningún gesto.

Ratón

Acción	Qué hace
botón izquierdo arrastrando	rotar
botón derecho arrastrando	desplazarse lateralmente
botón central arrastrando / rueda	acercarse y alejarse
clic sobre un cuerpo	enfocarlo y abrir su ficha completa
doble clic	capturar el puntero; Esc lo suelta
Shift + rueda	empuje

Táctil (móvil y tableta)

Gesto	Qué hace
un dedo arrastrando	rotar
dos dedos pellizcando	acercarse y alejarse
dos dedos arrastrando	desplazarse
toque sobre un cuerpo	enfocarlo y abrir su ficha

Teclado

Tecla	Acción
← → ↑ ↓	rotar la vista
W A S D	desplazarse · R / F subir y bajar
Q / E alabeo · Z nivelar	Shift turbo · Ctrl frenar · Espacio parada total
T / Y	encuadrar todo / encuadrar el objetivo
Ø	vista inicial · O cambiar de modo · G seguir
I	qué es este escenario · C ficha del objetivo
L rayo de luz · P sonda	2 2D/3D · H ocultar interfaz

El empuje se **adapta a la distancia del objeto más cercano**: al lado de un planeta vas fino, en el vacío interplanetario vuelas.

Modo órbita: la cámara gira alrededor del cuerpo seleccionado. Arrastra para girar, rueda para acercarte. Clic en cualquier cuerpo del visor para enfocarlo.

Panel

- **Curvatura** — profundidad del pozo, dureza del gradiente, realce de masas pequeñas, densidad y alcance de la retícula, opacidad. Los mandos de alcance, velocidad, tamaño y profundidad son **logarítmicos**: hacen falta, porque la escena va de 6 UL (un púlsar binario) a 3 millones (un cuásar).
- **Simulación** — velocidad en días por segundo (de 0,002 a 20.000), exageración de tamaños, **precesión relativista (1PN)** con su factor de exageración, presets de **calidad** (auto / alto / medio / bajo), pausa, reinicio, rayos de luz, sondas, **captura PNG** y **enlace** con todo el estado metido en la URL — útil para volver luego o para pasárselo a alguien ya montado.
- **Laboratorio de masa** — solo en ese escenario. Sube la masa y el objeto va cambiando de naturaleza solo (roca → supertierra → gigante gaseoso → enana marrón → estrella → enana blanca → estrella de neutrones → agujero negro), con su radio, densidad, gravedad superficial, velocidad de escape y corrimiento al rojo.
- **Universo conocido** — catálogo de **82 objetos reales**: planetas y lunas del sistema solar, la secuencia estelar completa (de Próxima Centauri a R136a1), gigantes y supergigantes, variables y cefeidas, enanas blancas, púlsares y magnetares, diez agujeros negros (de Gaia BH1 a Phoenix A), *doce exoplanetas y objetos de gran escala. Buscador y ficha con el radio de Schwarzschild, la compacidad y la órbita real cuando se conoce. Al abrir una ficha aparece el insertor: eliges alrededor de qué cuerpo, a qué distancia (con atajos: su órbita real, entre Marte y Júpiter, zona habitable, al borde), la excentricidad, la inclinación, si es retrógrada, y si entra en órbita, como errante (pasa una vez y se va) o en reposo* (cae en línea recta). Te dice el periodo resultante antes de insertar.*

- **Análisis del sistema** — energía total y momento angular con su **deriva** desde el arranque (un integrador simpléctico los conserva; si la deriva pasa de 10^{-3} el paso de tiempo se queda corto, y se marca en rojo), tabla de elementos orbitales (a, e, i, periodo) de cada cuerpo, **resonancias de movimiento medio** detectadas, y la **curva de rotación**: velocidad orbital frente al radio, calculada con toda la masa y solo con la visible. Las dos curvas separándose *son* el argumento de la materia oscura.
 - **¿Qué pasa si...?** y **Cuerpos** — ver la sección de arriba.
-

Lo que es real y lo que está exagerado

Real: masas, radios, distancias, órbitas (integrador Verlet con masas reales; la Tierra tarda 365,2 días sin ningún ajuste), radios de Schwarzschild, relaciones masa-radio, energías de impacto, forma de las geodésicas de luz y de las trayectorias de los fotones alrededor de un agujero negro.

Exagerado a propósito: la **profundidad del pozo**. El radio de Schwarzschild real del Sol son 2,95 km — tres millonésimas de unidad de la escena. Sin exagerarlo no se vería absolutamente nada. El HUD siempre muestra el valor real al lado del visual.

Acelerado: la espiral de la fusión de agujeros negros y la velocidad de las ondas gravitatorias, para que quepan en unos segundos.

Rendimiento

Toda la deformación se calcula en la **GPU** (vertex shader): la CPU no toca ni un vértice de la malla por fotograma. Además:

- techo duro de 520.000 vértices en la retícula, con reducción automática de densidad;
 - trazas de longitud fija con buffer que se compacta (no crecen nunca);
 - degradación automática si bajan los fps: primero pasos del trazador de geodésicas, luego resolución, luego densidad de malla — y se recupera sola al subir;
 - el bucle se detiene por completo con la pestaña en segundo plano;
 - el servidor es estático puro: lee el fichero y lo manda.
-

Aula: usarlo en clase

Botón **Aula**, abajo a la derecha. **99 lecciones guiadas** —352 pasos y 95 preguntas, que recorren 227 de los 491 escenarios— repartidas en tres niveles, con **buscador** y **filtro por nivel**: con noventa y una, una lista corrida no hay quien la maneje. El buscador mira el título, el resumen y el texto de los pasos, y no le importan las tildes: «cumulo» encuentra «cúmulo».

Primaria — 22 lecciones. Por qué no se cae la Luna · Quién manda en el sistema solar · Los ocho planetas de cerca · La Luna más cerca y más lejos · No todas las estrellas son iguales · Las piedras que sobraron ·

Cuando dos cosas chocan · ¿Y si la Tierra estuviera más cerca del Sol? · La Tierra de mudanza · ¿Y si tuviéramos dos lunas? · Dos Tierras en la misma órbita · Quitar un planeta y ver qué falla · ¿Y si nuestro Sol fuera otro? · Las lunas de Júpiter · Por qué Saturno tiene anillos · Planetas sin estrella · Lo que hay más allá de Neptuno · ¿Hay un noveno planeta? · Mercurio, el más rápido · Qué es un agujero negro · El zoo del universo · El sistema solar es casi todo vacío.

Secundaria — 35 lecciones. Las tres leyes de Kepler medidas · La zona habitable · Las mareas y la Luna que se aleja · Qué cambia cuando cambias la masa · Velocidad de escape · Resonancias · Los cinco puntos de Lagrange · El límite de Roche · Otros sistemas de verdad · ¿Nos protege Júpiter? · Órbitas alargadas · Estrellas que van de dos en dos · De dónde salió la Luna · Inclinación y sentido de giro · La zona habitable según el tipo de estrella · La misma órbita, distintas estrellas · Justo por debajo y justo por encima · Troyanos · Planetas con dos soles · Mundos de agua · Las Tierras más parecidas · Sistemas apretadísimos · Los primeros exoplanetas · Ver planetas naciendo · Júpiter de mudanza · Multiplicar la masa · Órbitas circulares · Qué pasa según lo que choque · Planetas chocando · Fobos · Siempre la misma cara · Visitantes de fuera · Cuando las dos son igual de grandes · Una hipótesis que se descartó.

Bachillerato — 34 lecciones. Qué es un agujero negro · Cómo nace una estrella · Por qué el sistema solar no se deshace · Donde Newton se queda corto · Agujeros negros supermasivos · Ondas gravitacionales · Materia oscura · El problema de tres cuerpos · Cuándo un sistema aguanta · La vida y la muerte de una estrella · Los planetas se mueven de sitio · De la nube al sistema solar de hoy · Sistemas de muchos cuerpos · Mundos imposibles · Kozai-Lidov · Agujeros negros por décadas de masa · Cómo se detecta un agujero negro · Estrellas de neutrones · De dónde viene el oro · Los agujeros negros que faltaban · Estrellas expulsadas de la galaxia · Binarias que se transfieren masa · Choques de materia degenerada · Encuentros estelares · Coreografías periódicas · Megaestructuras · Órbitas que parecen imposibles · Agujeros de gusano y masa negativa · Sistemas de tres, cuatro y cinco estrellas · La misma nube, distintos finales · Impactos y la historia de la vida · Cómo se mide lo que no se puede tocar · El futuro del sistema solar · La escala más grande de todas.

Cada paso carga su escenario solo, dice **qué mirar**, y muchos preguntan. La regla con la que están escritas las preguntas es que **todas se responden comprobándolas aquí dentro**, no de memoria: si al fallar te dice «no», el texto que viene detrás te señala el botón o la ficha donde se ve. Al terminar sale el recuento de aciertos.

Para dar clase con esto: proyectar el escenario, ir pasando los pasos, y dejar que sean los alumnos quienes toquen los mandos para contestar. Funciona igual en la pizarra digital, en el portátil y en el móvil de cada uno.

En el móvil

Está pensado para el teléfono, no solo «reducido». Lo que cambia:

- **El panel es una hoja que sube desde abajo** y arranca **plegada**, para que lo primero que se vea sea el cielo. Se abre con el botón de rayas de abajo a la izquierda, y se cierra con el mismo botón.
- **Arranca en modo simple:** se ven las cinco secciones que se usan siempre. Con «**Ver todas las opciones**» aparece el resto. Es la respuesta a tener once secciones abiertas a la vez, que en una pantalla

de teléfono no hay quien las maneje.

- **El mando de cámara está escondido** detrás de un botón redondo, porque con el dedo no hace falta: se **arrastra para girar** y se **pellizca para acercar**. Quien lo prefiera con botones lo saca y se queda fuera.
- El HUD se va arriba a la derecha y se queda con lo esencial.

Todos los botones, casillas y deslizadores están a 40 px o más, que es lo que hace falta para acertarlos con el dedo.

Subirlo a un servidor

Es HTML, CSS y JavaScript estáticos: **no hace falta Node en el servidor** (`server.js` es solo para trabajar en local). Se sube la carpeta entera menos `dev/` y `capturas/`.

El único requisito de verdad es que el servidor mande los `.js` con `Content-Type: text/javascript` — son módulos ES y, si llegan como `text/plain`, el navegador los rechaza y la pantalla se queda en «Generando el espacio-tiempo...».

El subdominio previsto, y el que ya está escrito en las etiquetas de la página, es `orbitas.gmartos.es`. Los detalles (qué tocar si se cambia, compresión, caché y la comprobación posterior) están en la sección 9 de `DOC-TECNICO.md`.

Documentación

- `DOC-TECNICO.md` — arquitectura, ficheros y shaders.
- `DOC-FISICA.md` — el modelo físico, las fórmulas y qué aproximación usa cada cosa.
- `guia.html` — la misma explicación en una página web, pensada para enlazar y compartir.

Orbit Universe — documentación técnica

Aplicación web de una sola página. **Sin build, sin bundler, sin** `npm install` : módulos ES nativos + un `importmap` . Three.js r169 va incluido en `vendor/` .

1. Árbol

orbit universe/	
index.html	cáscara: canvas, panel vacío, HUD, leyenda, importmap
server.js	servidor estático de 60 líneas, sin dependencias
css/app.css	toda la interfaz
js/	
main.js	clase App: escena, bucle, escenarios, acciones
core/	
units.js	sistema de unidades y formateadores
warp.js	★ el modelo de curvatura (GLSL + espejo en JS)
physics.js	cuerpos, integrador Verlet, colisiones, fotones, sondas
astro.js	relaciones masa-radio y clasificación de objetos
orbital.js	★ puntos de Lagrange, Hill, Roche, zona habitable, elementos orbitales, resonancias, invariantes y curva de rotación
view/	
materials.js	ruido fBm, materiales de línea y de traza
grids.js	retícula 3D, cáscaras equipotenciales, lámina 2D
bodies.js	estrellas, planetas, gigantes, halo, etiquetas
blackhole.js	★ trazador de geodésicas nulas (post-proceso)
starfield.js	fondo estelar procedural
trails.js	trazas de longitud fija
markers.js	L1-L5, esfera de Hill, límite de Roche, zona habitable
lensing.js	lente analítica (queda como alternativa ligera)
controls/flight.js	vuelo libre 6 GDL + modo órbita
ui/panel.js	construcción del panel, catálogo, laboratorio, HUD
data/	
scenarios.js	primer tomo: los escenarios fundacionales, escritos uno a uno, y el enganche de los otros tres tomos
scenarios2.js	segundo tomo: variantes "y si", sistemas planetarios reales de catálogo, estrellas múltiples, compactos, choques y teóricos
scenarios3.js	tercer tomo: FAMILIAS generadas (matriz de choques, agujeros negros por décadas de masa, resonancias, rosetas, cúmulos, discos)
scenarios4.js	cuarto tomo: familias CON UMBRAL (Gladman, Kozai, Holman-Wiegert, Roche, escape) y más exoplanetas
scenarios5.js	quinto tomo: NACIMIENTO de un sistema, de la nube de gas al disco protoplanetario
catalogo.js	82 objetos reales del universo conocido
lecciones.js	el aula: primer tomo y ensamblado
lecciones2.js	segundo tomo de lecciones
lecciones3.js	tercer tomo de lecciones
vendor/three/	build + addons de postprocesado
guia.html	página de texto indexable (1.229 palabras)
robots.txt	manifest.webmanifest
sitemap.xml	og.png icono-*.png
dev/	herramientas de comprobación, NO se suben:
humo.py	prueba de humo (capturas en capturas/humo/)
barrido.py	recorre los 491 escenarios buscando NaN y errores
visual.py	tres capturas de cada escenario y las mide
cuenta.py	guardián del catálogo contra dev/catalogo.json
tipos.py	audita el tipo de cada cuerpo (polvo, roca, estrella)
fisica.py	energía, momento angular e inversión temporal
saltos.py	whatif.py
capas.py	colapso.py

movil.py
icono.py

cuatro tamaños de teléfono: desborde y toque
genera icono-512.png e icono-180.png

2. Unidades

Magnitud	Unidad	Nota
Longitud	UL = 10^9 m (un millón de km)	radio solar 0,6957 · órbita terrestre 149,6
Tiempo	día = 86.400 s	
Masa	kg	reales, sin escalar

Con esas unidades la aceleración newtoniana es $a = GU \cdot M / r^2$ con $GU = 4,98213 \cdot 10^{-28}$ y $c = 25.902,068$ UL/día. La Tierra cierra su órbita en 365,2 días sin ningún ajuste: es la comprobación de que la aritmética cuadra.

3. El modelo de curvatura (core/warp.js)

La lámina 2D dice: *un punto a distancia r se hunde de forma proporcional a M/r* . Ese hundimiento no es una dirección real del universo; es un truco para dibujar en un papel algo a lo que le sobra una dimensión.

En 3D ese mismo hundimiento se convierte en **contracción radial**:

$$r' = r \cdot (r_f / (r_f + R_v)) ^ w \quad r_f = \max(r, R_{\text{cuerpo}})$$

- lejos: $r' \approx r - w \cdot R_v$ → cae como el potencial, igual que la lámina;
- cerca: $r' \rightarrow 0$ → la malla se estrangula: el embudo;
- r_f está acotado por el **radio del cuerpo**, y ahí está toda la diferencia entre el Sol, una enana blanca, una estrella de neutrones y un agujero negro.

$R_v = \text{warp} \cdot (\mu/\mu_\odot)^\gamma$ es el radio de Schwarzschild **visual**. $\gamma < 1$ realza los cuerpos pequeños para que la Tierra también deje huella.

Lo que se ve no es el desplazamiento, es la deformación

Primer intento fallido: colorear por `|desplazamiento|`. No funciona, porque a distancias grandes el desplazamiento tiende a una **traslación uniforme** ($w \cdot R_v$), y una traslación uniforme no es curvatura: la escena entera se encendía por igual.

Lo que se dibuja es la **compresión local de la celda**:

```

kt = (rf/(rf+Rv))^w           estiramiento tangencial
kr = kt * (1 + w·Rv/(rf+Rv))  estiramiento radial
comp = max(1 - kt, kr - 1)

```

$1 - kt$ cae como $1/r$, igual que el gradiente del potencial, así que el espacio lejano se apaga solo y la escena deja de ser una sopa de líneas.

Modo 2D: paraboloide de Flamm

En modo lámina se usa la incrustación exacta de Schwarzschild, $z(r) = 2 \cdot \sqrt{rs \cdot (r - rs)}$, invertida para que el embudo baje. No es una curva inventada: es la geometría de verdad de una rebanada ecuatorial.

Ondas gravitatorias

Cada fusión emite un pulso: un frente esférico con envolvente gaussiana y una oscilación senoidal, con amplitud que decae como $1/r$ y en el tiempo. Se suma a la posición del vértice y también al canal de "deformación", así que el frente se ve pasar iluminando la malla. Hasta 3 ondas simultáneas.

Todo esto vive en el vertex shader. La CPU sube 12 posiciones, 12 radios y 3 ondas por fotograma, y no toca ni un vértice más. `WarpField.apply()` es el espejo en JS de la misma función, y solo se usa para colocar los ~12 cuerpos y la cámara.

4. Agujero negro por trazado de geodésicas ([view/blackhole.js](#))

Es un pase de post-proceso. Para cada píxel:

1. se reconstruye el rayo de visión con la **base de la cámara** (nada de matrices: `rd = fwd + right·x·tanH·aspect + up·y·tanH`);
2. se descarta el píxel si el parámetro de impacto supera $14 rs$ — ahí la desviación es de décimas de píxel y no compensa;
3. se integra la geodésica nula en unidades de rs ($rs = 1$, $M = \frac{1}{2}$), con $a = -1,5 \cdot h^2 \cdot p / |p|^5$, que es la forma cartesiana de $d^2u/d\phi^2 + u = 3Mu^2$;
4. tres finales posibles: - $r < 1 \rightarrow$ **negro** de verdad, no llega nada; - cruce del plano del disco entre `r_in` y `r_out` $\rightarrow$ se **acumula** brillo (el disco es ópticamente fino, así que un mismo rayo lo cruza varias veces: de ahí el arco por encima del agujero, que es la cara de atrás del disco); - escapa $\rightarrow$ se devuelve el fotograma ya renderizado proyectando la dirección de salida sobre la pantalla (así se lensan de verdad las estrellas, la retícula y la estrella compañera).

Salen solos el anillo de fotones, el achatamiento del disco, el realce Doppler ($(1-\beta)^{-3}$) y el corrimiento al rojo gravitatorio ($\sqrt{1-1/r}$).

Coste: el pase hace *early-out* fuera del área del agujero, así que el gasto es proporcional al tamaño en pantalla, no a la resolución. Los pasos de integración (70–170) los baja y sube solo el gestor de calidad.

5. Física (`core/physics.js`)

- **Integrador:** velocity-Verlet con subpasos. El paso máximo se calcula de la órbita más corta del sistema dividida entre 80, y se recalcula al fusionar o eliminar cuerpos. Techo de 64 subpasos: si no llega, se ralentiza el tiempo en vez de perder precisión — nunca se atasca.
- **Suavizado de Plummer** en `soft` (el radio del cuerpo) para que la aceleración no explote.
- **Colisiones por barrido:** con pasos de horas, dos planetas a 30 km/s se cruzan sin tocarse si solo se compara la posición final. Se busca la **distancia mínima del segmento recorrido**. La fusión conserva masa y momento, el radio se combina por volumen y se emite el evento con la velocidad relativa y la energía cinética del impacto.
- **Radio de choque $\neq$ radio de dibujo:** con tamaños exagerados $\times 120$ la Luna quedaría dentro de la Tierra. Por defecto se usa el radio **real**; los escenarios de impacto declaran su `collideScale`, y las estrellas nunca se agrandan.
- **Espiral por radiación gravitatoria:** comprime la separación conservando $v \sim 1/\sqrt{a}$, con ritmo $\propto (\text{refSep}/r)^3$ — el "chirp" sale solo. Está acelerada varios órdenes de magnitud y así se dice.
- **Fotones:** en campo débil la desviación de la luz es exactamente el **doble** que la newtoniana. Se integra con esa aceleración y se renormaliza $|v| = c$ en cada paso. Usan las masas **visuales** ($Rv \cdot c^2/2$) para que lo que ves y lo que hace la luz coincidan.

5 bis. Geometría del campo (`core/orbital.js`)

Todo se resuelve en vivo con las masas y posiciones del instante:

- **L1, L2 y L3** salen de anular la aceleración a lo largo del eje que une los dos cuerpos en el sistema en rotación, $-\mu_1/d_1^2 \mp \mu_2/d_2^2 + \omega^2(x - x_{cm}) = 0$ con $\omega^2 = (\mu_1 + \mu_2)/R^3$. Se resuelve por **bisección** (70 iteraciones, robusta y sin derivadas). **L4 y L5** son exactos: triángulos equiláteros. Se marcan estables cuando $M_1/M_2 > 24,9599$.
- **Esfera de Hill:** $r_H = R (m/(3M))^{1/3}$.
- **Límite de Roche:** $1,26 R_1 (\rho_1/\rho_2)^{1/3}$ rígido y $2,44 R_1 (\rho_1/\rho_2)^{1/3}$ fluido.
- **Zona habitable:** límites conservadores escalados con la luminosidad, $L \sim M^{3,5}$.
- **Invariantes:** energía y momento angular. El halo entra en la energía potencial (es un potencial externo que tira de todos) pero no en la cinética ni en el momento, porque no se mueve. Sin esa distinción la energía "derivaba" un 6 % y el chivato daba una falsa alarma.
- **Resonancias:** se buscan razones de periodos cercanas a $p:q$ con enteros pequeños y se ordenan por error relativo. En el escenario de Ío-Europa-Ganímedes salen solas la 2:1, la 2:1 y la 4:1.

Los marcadores se dibujan con el **mismo material deformado** que la retícula, así que se curvan con la geometría en vez de flotar por encima de ella.

5 ter. Qué tamaño se dibuja cada cuerpo

Exagerar los tamaños es imprescindible —la Tierra son 6.371 km en una escena de cientos de millones— pero exagerarlos a lo bruto rompe la lectura de dos formas concretas, las dos reportadas por el usuario:

1. un planeta acaba dibujado **más grande que su estrella**;
2. dos cuerpos **se atraviesan en pantalla y no pasa nada**, porque el choque se mide con los radios físicos y el dibujo va exagerado.

`App.refrescaPrimarios()` recalcula dos veces por segundo (no cada fotograma: con 220 cuerpos son 24.000 pares) el radio de dibujo de cada cuerpo con reglas duras:

- ninguno pasa del **22 % de su distancia al primario**;
- ninguno que no sea estrella pasa del **2 % del alcance** (las estrellas, del 10 %);
- ninguno que no sea estrella **supera el 80 % de la estrella más pequeña** de la escena;
- **ningún par de esferas se solapa**: si se pisan, se encogen las dos en proporción, y nunca por debajo de su radio físico real.

Y hay que aplicarlo **con el tiempo, no solo al calibrar**: un par que arranca lejísimos acaba encima. Ese fue el fallo — se validaba con una captura por escenario.

La excepción inevitable: un satélite y su planeta

Ío orbita a **seis radios** de Júpiter. Si se quiere ver a Ío fuera de Júpiter, a Júpiter no se le puede exagerar más de seis veces... y entonces, a la escala del sistema solar, Júpiter es un punto. Es una contradicción de la realidad, no del programa.

Se resuelve dejándole al planeta su tamaño y **ocultando la esfera del satélite** cuando le cae dentro: el satélite se sigue viendo por su marcador, su nombre y su traza, pero no aparece una bola atravesando otra. Entre cuerpos **independientes** la regla se aplica entera, y por eso ahí **tocarse vuelve a significar chocar**.

Y lo mismo con las cáscaras y con el encuadre

Dos fallos de la misma familia, encontrados por la auditoría:

- Las **cáscaras equipotenciales** se dibujaban hasta el 42 % del alcance. En una comparativa —una fila de ocho estrellas— la esfera de la más masiva se tragaba a todas las demás y solo se veía una mancha. Ahora ninguna pasa de **la mitad de la distancia al vecino más próximo**.
- El **plano lejano** de la cámara estaba fijo en 400.000 UL. En las escenas grandes (Sagitario A, *TON 618*, *el Planeta Nueve*) todo* quedaba detrás de él y la pantalla salía vacía. Ahora sale del alcance de la escena; el buffer de profundidad es logarítmico, así que el rango no da problemas.
- Las **comparativas congeladas** orbitan el centro de la escena y encuadran la fila entera, en vez de girar alrededor del cuerpo más masivo y sacar al resto de cuadro.
- El **giro automático** de la cámara viene apagado: una vista que se mueve sola mientras intentas leer algo es lo contrario de lo que hace falta.

Cómo se comprueba

`dev/auditoria.py` recorre los 59 escenarios y los muestrea en **cuatro instantes** distintos, midiendo por escenario: pares solapados, cuerpos mayores que su estrella, posiciones no finitas, cuerpos fuera de encuadre y deriva de la energía. Guarda tres capturas de cada uno y un JSON con todo. Una foto suelta no vale: los fallos que importan salen con el tiempo.

5 quater. El editor de órbitas

`App.moverAOrbita(id, {a, e, inc, retro, hostId})` recoloca un cuerpo a otra distancia de su anfitrión **conservando la dirección en la que estaba y el sentido de giro**, y le da la velocidad que corresponde a la órbita nueva (colocándolo en el periastro). A partir de ahí no hay guion: la simulación decide si se queda tranquilo, entra en resonancia, se cruza con un vecino o sale despedido.

Está expuesto de dos formas:

- en la **ficha de cualquier objeto**, con un campo de semieje, excentricidad, inclinación añadida, sentido de giro y atajos calculados sobre la escena que hay delante (−20 %, ×2, "zona habitable", "entre Venus y la Tierra", "más allá de Neptuno");
- como orden de escenario: `orbita:<cuerpo>:<UA>[/e[/inc]]`, para los botones de "¿y si...?".

Anfitrión ≠ quien más tira

`primarioDe` devuelve el cuerpo con mayor μ/r^2 , que es quien ejerce más fuerza. Para el editor eso **no** vale: el Sol tira de la Luna 2,2 veces más fuerte que la Tierra, y aun así la Luna orbita a la Tierra. `anfitriónDe` resuelve la jerarquía de verdad: entre los cuerpos más masivos a los que el cuerpo está **ligado** (velocidad relativa por debajo de la de escape), el más cercano. Sin esto, el editor ofrecía mover la Luna alrededor del Sol.

La unidad del campo también cambia con la escala: UA para un planeta, millones de kilómetros para un satélite. "0,0025581 UA" no le dice nada a nadie sobre la Luna.

5 quinquies. Muestreo de las trazas

Las trazas **no** se muestrean por tiempo, se muestrean por **distancia recorrida**: `Trail.push(v, minSep)` descarta el punto si está a menos de `minSep` del anterior, y `minSep` es la distancia al anfitrión dividida por 150. Así cada vuelta lleva siempre unos 150 puntos, vaya la simulación a un día por segundo o a mil.

Además se muestrea **dentro de los subpasos del integrador**, no una vez por fotograma:

`Integrator.step(dt, subpasos, onSub)` llama a `onSub` después de cada subpaso. Y el tope de subpasos depende de la carga (240 con menos de 30 cuerpos, 96 hasta 90, 64 por encima); al llegar al tope se recorta `dt`, así que lo que baja es la velocidad de la simulación, nunca la precisión.

Por qué: con muestreo por fotograma y a 900 días por segundo, la Luna daba treinta y tres vueltas por segundo y caía **menos de un punto por órbita**. En pantalla no salía una elipse, salía una telaraña de rectas

cruzadas. Medido después del arreglo, el hueco angular máximo entre puntos consecutivos es de 4° a cualquier velocidad entre 0,5 y 900 días por segundo.

5 sexies. El buscador de escenarios

Con 467 escenarios un desplegable a secas no sirve. `Panel.rellenaEscenarios()` reconstruye el `<select>` con lo que pasa el filtro, conservando las cabeceras por tipo y su recuento. La búsqueda es por palabras sueltas, sin acentos ni mayúsculas (`normalize('NFD')` y fuera los diacríticos), y mira clave, título, explicación y grupo: "marte cerca" encuentra los que llevan las dos palabras. El escenario cargado se mantiene siempre en la lista aunque no case, para que el desplegable no mienta sobre lo que se está viendo.

5 septies. Familias con umbral, y cómo se comprueban

Varias familias de `scenarios4.js` recorren un parámetro que cruza un umbral físico exacto. La promesa que hace su texto **está medida**, no supuesta, con `dev/umbrales.py` y con las sondas de `dev/`:

Familia	Umbral	Medido
Separación de Hill (Gladman 1993)	$2\sqrt{3} \approx 3,46$	A 2 y 3 radios las órbitas acaban cruzándose (la interior llega a 1,04 y la exterior baja a 0,99); a 4 y 10 no se mueven ni un 2 % en mil años
Trojanos en L4	$M1/M2 > 24,96$	8:1 y 15:1 expulsan el troyano a miles de veces la distancia; 24:1 oscila entre 46° y 80°; desde 26:1 se queda clavado en 60,0°
Velocidad de escape	100 %	50 % y 95 % ligados; 100 %, 105 % y 200 % escapan

Dos avisos que salieron de medir:

- **El montaje tiene que ser exacto o la familia no enseña nada.** La primera versión de los trojanos ponía la velocidad de L4 usando solo la masa de la estrella principal e ignorando la del compañero. Un error del 1 al 2 % en la velocidad, y con él se iban también los casos que deberían ser estables: la familia entera enseñaba el fallo del montaje, no el umbral. Ahora los tres cuerpos giran alrededor del baricentro con la velocidad angular exacta y L4 es el vértice del triángulo equilátero.
- **Estas inestabilidades tardan miles de vueltas.** A la velocidad de un escenario normal no se ve nada y parece que el umbral no existe. Por eso estas familias arrancan ya muy aceleradas (30.000 y 60.000 días por segundo) y el texto lo dice.

5 octies. Tamaños y encuadre: todo en píxeles de pantalla

Este bloque se reescribió entero el 29/08/2026 porque **casi todos los escenarios se veían mal**: una bola blanca en el centro y nada más. La causa no era una, eran cuatro, y ninguna se veía en las mediciones numéricas — sólo mirando las capturas.

Las cuatro causas, medidas

Qué se veía	Causa real	Antes	Ahora
Una bola blanca se comía el sistema	El pozo de curvatura se calibraba como el 7 % del alcance. En el sistema solar eso son 605 UL de radio, y Mercurio orbita a 58: los cuatro planetas interiores estaban dentro de la bola	605 UL	13,8 UL
Los planetas no aparecían	La Tierra se dibujaba a 0,62 px : la regla de no-solape la encogía 14 veces para no tocar a la Luna, porque primarioDe decía que el primario de la Luna era el Sol	0,62 px	9 px
Los agujeros negros, enormes	No pasaban por ninguna regla de tamaño: el suyo salía de la exageración de la curvatura, sin techo	32-45 px	13 px
El cuásar y la disrupción por marea llenaban el cuadro	El encuadre contaba el horizonte pero no el disco de acreción , que llega a 30 radios	78 % del cuadro	10 %

La regla, ahora

Todo se decide en **píxeles de la pantalla**, no en unidades de mundo. Es la única forma de que la respuesta sea la misma en un sistema de 0,06 UA de ancho y en uno de 70.

Cada cuerpo tiene un tamaño deseado y cuatro techos, y gana el más bajo:

1. no taparse su propia órbita — 20 % de la distancia a su anfitrión;
2. no comerse al vecino — 35 % de la distancia al cuerpo más cercano que no sea su planeta ni su luna;
3. no taparle la órbita a su luna — 28 % de la distancia a su satélite más cercano, **sólo para estrellas y objetos compactos** (que un planeta se trague a su luna es la contradicción aceptada; que una estrella se trague sus planetas, no);
4. no acaparar la pantalla — 5 % del alto de la ventana.

Por debajo, un **suelo de 2,5 px**, y por debajo de todo el **radio físico real**: si un cuerpo es de verdad más grande que su techo, se dibuja como es. Eso es información.

La exageración ya no es un número global calculado a ojo (**rscale** daba 1,03 en el sistema Tierra-Luna, o sea ninguna): se calcula cuánto hay que multiplicar para que el **mayor de los cuerpos no estelares llegue a 9 px**, conservando enteras las proporciones entre cuerpos. Un agujero negro no usa ese factor — su horizonte es cientos de veces menor que cualquier otra cosa y lo convertía en un monstruo — sino un objetivo propio de 13 px.

Después: la jerarquía (nada que no sea estrella pasa del 45 % del tamaño **angular** de la estrella más pequeña) y la resolución de solapes.

El encuadre

- **El centro** no es el centro de masas geométrico cuando hay un cuerpo que domina: en el sistema solar la media de posiciones cae a mil UL del Sol, arrastrada por los gigantes, y el sistema interior quedaba en un rincón. Si un cuerpo tiene más de diez veces el **mu** del siguiente, se centra en él.

- **El alcance** no es el cuerpo más lejano: las distancias de un sistema planetario son casi logarítmicas y Neptuno está a 78 veces la órbita de Mercurio. Cuando el más lejano supera ocho veces la mediana, se encuadra hasta **tres veces la mediana**.
- **Cuenta el disco de acreción**, no sólo el horizonte, con un techo (60 veces el núcleo) para que la cámara no se vaya tan lejos que el objeto quede en un punto.
- La distancia de cámara es **2,0 r** y no 2,6: con 58° de campo hace falta 1,8 r para llenar el alto, y el margen de sobra dejaba toda la escena en el suelo de 2,5 px.

Lo que sigue sin ser perfecto, y por qué

- **Discos y anillos muy poblados** (240 partículas) dejan algún par de esferas tocándose. Con esa densidad, la separación media entre partículas es menor que el tamaño mínimo visible: o se ven las partículas, o no se tocan, pero no las dos cosas.
- **Un agujero negro sin nada que lo ilumine sale negro**. Es correcto y es inútil de mirar. Se ve por el anillo de estrellas lensadas y poco más.
- **Los cúmulos pequeños se van de la pantalla** al cabo de unos segundos. Eso es real: un cúmulo de 25 estrellas no está ligado y se evapora.

Cómo se comprueba

`dev/visual.py` captura **tres tomas de cada escenario**, separadas en el tiempo, mide el PNG resultante (no números de la simulación) y genera `capturas/visual/index.html` con las 1.401 imágenes, poniendo primero y en rojo las que están mal. Los fallos que importan no se ven en una foto suelta: un escenario puede arrancar perfecto y estar vacío diez segundos después.

Y la lección de método: la primera versión de esa medición contaba «píxeles brillantes» y los llamaba «cuerpos». Las líneas de la retícula también brillan, así que medía la malla y daba 12 % con la pantalla llena de nada. Una métrica mal nombrada es peor que no medir: da confianza falsa.

5 nonies. El encuadre se MIDE, no se calcula

Colocar la cámara con una fórmula abierta no funciona, y se intentó de siete maneras: «dos veces el alcance», «tres veces la mediana», «el 35 % del span»... El problema es doble. Primero, la fórmula tiene que acertar a la vez en un sistema Tierra-Luna de media UL, en el sistema solar de 4.500 y en una galaxia de un millón. Y segundo, y peor: **el tamaño con el que se dibuja cada cuerpo depende de donde este la cámara**, así que cualquier fórmula que use `visualRadius` se realimenta.

Medido con la fórmula abierta: en el escenario de la Tierra sola la cámara acabo a dos mil radios terrestres, con la Tierra de nueve píxeles en mitad de la nada; y en el sistema solar completo, todo el sistema quedaba en un nudo de tres píxeles.

Ahora no hay fórmula, hay **medida**: se coloca la cámara, se PROYECTAN los cuerpos a la pantalla, se mira cuanto ocupan de verdad y se corrige la distancia. Cinco iteraciones bastan y funciona a cualquier escala porque no supone nada sobre ella.

- Objetivo: que los cuerpos quepan en el **62 % del alto** de la pantalla.

- Con **seis cuerpos o menos** entran todos. Con más, el **percentil 60**: en un sistema planetario las distancias son casi logarítmicas (Neptuno esta a 78 veces la órbita de Mercurio) y meter al último deja a los demás en un punto.
- **Un solo cuerpo** no se puede medir (todos los puntos caen en el centro): ahí se coloca con el radio físico y el alcance de la escena, y contando el disco de acreción si lo tiene.

5 decies. El pozo se mide contra lo que orbita

warp decide el tamaño del pozo, que es lo que deforma la retícula y lo que dibujan las cascaras. Estaba atado al alcance de la escena (7 %) y por eso casi todo se veía como una bola blanca: en el sistema solar salía un pozo de **605 UL con Mercurio orbitando a 58**.

Se mide contra la distancia del cuerpo dominante a su **satélite más cercano** (30 %). Y si nadie le orbita — tres estrellas iguales, un cúmulo, una binaria pareja — contra su **vecino más cercano**: sin ese segundo caso, el problema de los tres cuerpos salía con un pozo de 125 UL y los tres soles separados 82, o sea los tres dentro de la misma mancha.

5 undecies. Un agujero negro no ocupa lo que su horizonte

El trazado de geodésicas le dibuja la sombra, el anillo de fotones y, si lo tiene, un **disco de acreción que llega a treinta veces el radio**. Dándole trece píxeles al horizonte, el conjunto medía casi cuatrocientos y tapaba la escena. Lo que se fija es el tamaño del CONJUNTO (unos 100 px) y de ahí se deduce el del horizonte. El encuadre también cuenta el disco, no solo el horizonte.

Y el resplandor de las estrellas iba a 2,8 veces el radio; con el bloom por encima, una estrella de 25 px de radio acababa siendo un borron de 195 px de ancho. Ahora, 1,5.

5 duodecies. Viaje en el tiempo

El mando de velocidad llega a 20.000 días por segundo: 55 años por segundo. Para ver un millón de años harían falta cinco horas mirando la pantalla. Hay botones de salto de 1, 100, 10.000, un millón y diez millones de años, en los dos sentidos.

Saltos cortos: se integra de verdad, con el mismo integrador y el mismo paso seguro que el bucle normal, así que el resultado es idéntico a haberlo visto pasar, perturbaciones incluidas. El último paso se ajusta para caer exacto en el lapso pedido.

Saltos largos: un millón de años con el paso seguro del sistema solar (0,32 días) son mil cien millones de pasos. No caben. Y agrandar el paso **no es «menos preciso», es destruir la órbita**: con pasos de 913 días una órbita de 365 no sobrevive, el planeta sale despedido, y el paso seguro que devuelve el sistema después es de 1,7 billones de días, o sea que ya no hay órbitas. Medido.

Así que para lapsos largos se hace lo que hacen las efemerides de verdad: **propagación kepleriana** con los coeficientes de Lagrange f y g y la anomalía excentrica, cuerpo por cuerpo alrededor de su anfitrión. Es

exacto para dos cuerpos y muy bueno en un sistema jerárquico. Lo que NO recoge son las perturbaciones entre planetas, y **se avisa en pantalla**: un salto que parece bueno y no lo es, es peor que no tenerlo.

Dos detalles que costaron:

- Los anfitriones se propagan primero, así que al llegarle el turno a un satélite su planeta YA se ha movido. Restando la posición NUEVA del planeta al satélite VIEJO sale un vector relativo que no existió nunca: la Luna acababa a nueve millones de UL de la Tierra. Hay que sacar el vector relativo de una foto del estado anterior.
- La anomalía media hay que reducirla módulo el periodo (`días % T`) o Newton no converge: dar mil millones de vueltas deja el cuerpo donde estaba.

Comprobado: ida y vuelta de 10 mil, 1 millón y 10 millones de años devuelve la Tierra a 0,9833 UA y la Luna a 0,3647 UL, exactamente los valores de partida.

Las fechas

`Date` de JavaScript se rompe pasados unos 273.000 años (su rango útil son $8,64 \times 10^{15}$ milisegundos): salía «Invalid Date». `simDate` dice ahora lo que se entiende según la escala: fecha de calendario dentro de +-300 años, «año 12.000» hasta el millón, «dentro de 4,38 M de años» por encima y «hace 10,089 mil M de años» pasados los mil millones.

5 terdecies. El nacimiento de un sistema

Trece escenarios que van de una nube de gas al sistema solar de hoy, y otras cinco nubes de masas distintas.

Nada esta guionizado: se sueltan los grumos con la masa, el tamaño y el giro que toca y la gravedad hace el resto. El pozo del centro crece porque crece la masa que hay ahí, no porque nadie lo anime.

Tres cosas hay que acertar para que una nube colapse de verdad y no se quede flotando:

- **Sub-virial.** Con la velocidad circular completa la nube se queda como esta; con una fracción, cae. Una nube molecular real está justo así, sostenida a medias por turbulencia y campos magnéticos.
- **Grumos gordos.** Un grumo de una centésima de masa solar mide cientos de UA, no lo que una estrella. Poniéndoles radio de estrella no se tocan nunca: se cruzan, se dispersan y la nube se evapora en vez de formar nada.
- **Un poco de rotación.** Sin ella todo cae al centro y sale una estrella sola; con ella sobra momento angular y queda disco, que es de donde salen los planetas.

Y una de escala: el tiempo de caída libre va como el radio elevado a 1,5. Con las 10.000 UA de una nube molecular típica harían falta cien mil años y no se ve pasar nada; con 1.400 UA son diez mil años y el proceso entero cabe en medio minuto de reloj. Las nubes de estos escenarios son a propósito más compactas que las de verdad, y el texto lo dice.

Comprobado con `dev/colapso.py`, que mide si colapsan en lugar de suponerlo:

escenario	cuerpos	el mayor se queda con	veredicto
origen-1-nube	120 → 12	4,8 % → 81 %	colapsa y forma una estrella
nube-tipo-sol	100 → 5	→ 90 %	una estrella con restos
nube-enana-roja	60 → 3	→ 95,5 %	una enana roja
nube-masiva	140 → 4	→ 68,6 %	una estrella masiva
nube-cumulo	180 → 62	→ 10,5 %	un cumulo: ninguna domina

El radio que contiene la mitad de la masa cae de 119.000 a 2.280 UL, y el radio de curvatura del cuerpo central sube de 19.800 a 26.900. Eso último es literalmente «como va aumentando la gravedad», medido.

5 quaterdecies. Polvo, círculos de distancia y la ficha del escenario

`kind: 'polvo'`

Un grumo de una nube molecular, una partícula de un anillo o un fragmento de cometa **no es un planeta**, y dibujarlos con la malla esférica y la textura de mundo daba escenas absurdas: doscientos planetas con continentes chocando entre sí cuando eso es polvo y gas.

El tipo `polvo` se dibuja como una mota difusa: sin superficie, sin relieve, **sin marcador y sin nombre** (doscientos rótulos de «Grumo 137» tapan la escena), y con **mezcla normal, no aditiva**. Ese detalle importa: el primer intento reutilizaba el material del resplandor, que es aditivo, y 120 motas superpuestas sumaban hasta blanco puro — el 42 % de la pantalla. En el sistema de tamaños el polvo va aparte: siempre entre 1,3 y 3,2 píxeles, exento de la jerarquía contra la estrella y **exento de la resolución de solapes**, porque una nube se solapa por definición.

`dev/tipos.py` recorre los 491 escenarios, mira los cuerpos que se generan de verdad (no el código: muchos salen de generadores) y marca lo que no cuadra: rocas dibujadas como planetas, enjambres de mundos idénticos, lunas que orbitan una estrella, gigantes gaseosos por debajo de diez masas terrestres, planetas por encima de trece masas jovianas y estrellas por debajo del límite de encendido.

Círculos de distancia

Seis anillos alrededor del cuerpo que manda, en distancias redondas de una escalera fija, rotulados **en UA y en millones de kilómetros**. Sin una regla en el suelo no hay forma de saber si lo que ves son dos décimas de UA o doscientas: la cámara puede estar a cualquier distancia y no hay con qué comparar. Se eligen las que caben entre el 1,5 % y el 95 % del alcance, así que funcionan igual en TRAPPIST-1 (0,002 a 0,1 UA) que en una nube molecular (50 a 2.000 UA).

Detalle del sistema de unidades que lo hace trivial: **1 UL = 1e9 m = un millón de kilómetros clavado**, así que el radio en UL ya es la cifra en millones de km.

La **zona habitable** pasa a estar encendida por defecto en todos los escenarios. Era un permiso por escenario (`hz: true`) y solo lo tenían veinte; en los otros cuatrocientos sesenta no había forma de saber si un planeta estaba donde puede haber agua líquida. Se apaga sola cuando no hay estrella o cuando la zona cae fuera de la escena.

La ficha del escenario y sus tiempos

El botón **(i)**, siempre visible, abre la ficha (también con la tecla **I**). Además del texto dice **qué hay** (cuántos cuerpos de cada clase, masa total, quién manda y con qué porcentaje) y **cuánto tarda cada cosa**: la vuelta más corta y la más larga con su periodo real y cuántos segundos de reloj cuesta verla a la velocidad actual, el tiempo de caída libre si la escena es una nube, y el aviso si está congelada.

Nada de eso está escrito: se calcula del sistema que hay delante. Por eso vale para los 480 y por eso no puede contradecir a la simulación — cambia si mueves un cuerpo, si quitas otro o si tocas la velocidad. El polvo se excluye de los periodos: «el Grumo 94 tarda 636 millones de años en dar la vuelta al Grumo 36» no le dice nada a nadie y además es una órbita que no existirá dentro de diez segundos.

5 quincecenas. El repaso a fondo: qué se probó y qué salió

Cinco superficies que no se habían tocado, cada una con su herramienta en `dev/`.

Los botones «y si...» (`dev/whatif.py`)

Son órdenes como `kill:luna` o `mass:sol:2`, y si el identificador no existe en ese escenario **el botón no falla: no hace nada**, sin un error en consola. Se pulsan todos, recargando el escenario entre uno y otro, y se comprueba que la huella del sistema (número de cuerpos, masas, posiciones, velocidades) ha cambiado.

76 botones, 0 rotos. El único que salió marcado es un falso positivo: en la precesión de Mercurio, el botón «Relatividad ×1.000.000» no cambia nada porque el escenario ya arranca con ese valor.

Las capas (`dev/capas.py`)

Una capa desconectada se marca en la casilla y no pasa nada. Se apaga cada una y se mide si el fotograma cambia, **con un control de ruido**: dos capturas seguidas sin tocar nada, para no confundir el temblor de la escena con el efecto de la capa.

Las 13 funcionan. Las que salían marcadas eran de la prueba, no del programa: sin tiempo corriendo no hay estelas que ocultar, sin agujero negro no hay nada que lентear y sin disparar un rayo no hay geodésica que dibujar. Cada capa se volvió a probar en un escenario donde Sí tiene que verse.

La física (`dev/fisica.py`)

Dos invariantes que no dependen de cómo se vea nada: la deriva de energía y la **simetría temporal** — correr 400 pasos hacia delante y 400 hacia atrás tiene que devolver los cuerpos a su sitio.

Ocho de diez escenarios vuelven con error relativo entre $1e-13$ y $1e-16$. Los tres que no vuelven se explicaron **quitando la causa y volviendo a medir**, no suponiendo:

escenario	tal cual	quitando la causa
Tierra-Luna	6,3e-3	1,4e-13 sin mareas
Precesión de Mercurio	3,4	2,7e-14 sin la corrección relativista
Cúmulo de 25 estrellas	0,10	dos estrellas se fusionaron (25 → 24)

Las tres son irreversibles **a propósito**: las mareas disipan energía, la corrección 1PN depende de la velocidad (y el escenario la exagera un millón de veces), y una fusión destruye información. La herramienta ya conoce esas tres excusas y no vuelve a dar la alarma por ellas.

El catálogo

Los **82 objetos** del catálogo se insertan sin un solo fallo ni un NaN.

Lo único que salió roto de verdad

55cnc no se veía: todos los cuerpos a 0,2 píxeles. 55 Cancri tiene los planetas entre 0,015 y 5,96 UA, cuatrocientos a uno. Como son seis cuerpos, el encuadre aplicaba la regla de «con pocos, entran todos», se iba a 1.688 UL, y entonces los toques de tamaño (no taparle la órbita a tu vecino, no taparle la órbita a tu planeta más interior) dejaban a la estrella en 0,4 píxeles.

Lo que decide el encuadre no puede ser **cuántos** cuerpos hay, sino el **rango**: ahora se meten todos solo si el más lejano no pasa de ocho veces la mediana. Medido después: estrella de 10,8 px y planetas de 4 a 4,9.

6. Rendimiento

Medida	Dónde
Toda la deformación en GPU	<code>warp.js</code> + <code>materials.js</code>
Techo de 520.000 vértices en la retícula, con reducción de densidad automática	<code>grids.js</code>
Menos líneas verticales que horizontales (el sistema es un disco)	<code>grids.js</code>
Trazas de longitud fija con compactado amortizado — no crecen jamás	<code>trails.js</code>
Desvanecido por edad calculado con el índice del vértice: 0 trabajo por fotograma	<code>materials.js</code>
Degradación automática: pasos del trazador → resolución → densidad, y recuperación	<code>main.js:adapt()</code>
Bucle detenido con la pestaña oculta	<code>main.js:loop()</code>
Buffer de profundidad logarítmico (de 1.700 km a 900 millones)	<code>main.js</code> + <code>LOGD_*</code>
Geometrías de esfera/quad compartidas por todos los cuerpos	<code>bodies.js</code>
Servidor estático puro, sin proceso ni plantillas	<code>server.js</code>

7. Detalles que costaron y conviene no repetir

- `#include <...>` de **Three.js** debe ir al principio de línea. Su parser es un regex `^[ \t]*#include`. Un `precision highp float; #include <common>` en la misma línea rompe el shader entero.
- **Nada de matrices en un `ShaderPass`** si se puede evitar. La primera versión del agujero negro usaba `inverse(P·V)` y salía todo negro porque `matrixWorldInverse` aún no estaba actualizado en ese punto del fotograma. Con la base de la cámara (`right`, `up`, `fwd`, `tanH`, `aspect`) no hay nada que pueda desincronizarse.
- **Cache-Control en los `.js`**: durante el desarrollo conviene ponerlo a `no-cache` o forzar recarga dura, o se depura una versión que ya no existe.
- El desplazamiento **no** es una buena señal visual (ver §3).
- `preserveDrawingBuffer: true` es imprescindible para poder capturar el fotograma con `toDataURL`. Cuesta algo de memoria, pero sin él la captura sale en negro.
- **Nunca anular `attribute.updateRanges`**. `WebGLAttributes` lo lee en cada subida (`attribute.updateRanges.length`), así que ponerlo a `null` revienta la aplicación entera... pero solo cuando el buffer de trazas se llena por primera vez, a los ~45 segundos de simulación. Lo correcto es `attribute.clearUpdateRanges()`. Por eso `dev/completo.py` deja la escena corriendo 60 segundos seguidos: los fallos que aparecen tarde no los caza una prueba de humo de tres segundos.
- **Nunca cortar un fichero por un índice**. Reescribir un bloque con `s[:i] + nuevo` se llevó por delante los 21 escenarios que venían detrás, sin un solo error de sintaxis ni de consola: el fichero seguía siendo

JavaScript válido y la aplicación seguía arrancando. Se sustituye siempre un bloque **acotado por sus dos extremos**, comprobando que los dos existen. Y después se pasa `dev/cuenta.py`, que compara la lista de escenarios que la aplicación sirve de verdad con la referencia guardada en `dev/catalogo.json` y nombra los que hayan desaparecido. Ocho segundos.

- **Ninguna velocidad se pone a ojo.** Las del Trapecio de Orión y las de los cúmulos estaban puestas a mano y las dos veces estaban mal: doscientas veces por debajo de la de escape en un caso y por encima en el otro, y en los dos el sistema se destrozaba antes de que diera tiempo a mirarlo. Toda velocidad inicial se **deriva del propio sistema** — de la masa que de verdad queda encerrada, del baricentro, de la velocidad angular exacta — y se comprueba midiendo, no mirando.
- **El paso máximo de integración hay que recalcularlo, no solo al cargar.** Depende de la distancia entre cuerpos, y esa cambia sola: en un cúmulo, dos estrellas que empiezan lejísimos acaban rozándose, y entonces el paso que era seguro deja de serlo y el integrador le regala energía a la pareja. Ahora se recalcula dos veces por segundo cuando hay 300 cuerpos o menos.
- **El alcance de la retícula tiene un tope duro de 4e6 UL** (`autoCalibrar`). Una galaxia a escala real lo supera por ocho órdenes de magnitud, y como el plano lejano de la cámara se deduce del alcance, TODA la escena queda detrás de él y la pantalla sale vacía sin un solo error. Las escenas galácticas van con las distancias comprimidas, igual que la de la Vía Láctea y Andrómeda.
- **La velocidad de un cúmulo hay que sacarla de la masa encerrada REAL**, contando las estrellas que se acaban de colocar, no de un perfil teórico que no corresponde al reparto generado. Con un Plummer que sobrevaloraba la masa interior en un factor 1,7 todas las estrellas salían por encima de la velocidad de escape y el cúmulo se evaporaba antes de que a nadie le diera tiempo a mirarlo.
- **Masa negativa:** `auto: false` **obligatorio.** La calibración automática deduce la exageración de la curvatura del pozo más hondo, y con masa negativa el radio de curvatura sale **negativo**: la contracción radial $r \cdot (r_f / (r_f + R_v))^w$ se va entonces a números absurdos. Medido en el escenario del agujero de gusano antes de arreglarlo: alcance de cuatro millones de UL y cuerpos dibujados a 3e13. Pantalla en negro, cero errores en consola. Todo escenario con masa negativa trae sus propios `span`, `warp`, `gamma`, `rscale` y `cam`.
- `zeroMomentum` **dividía por la masa total.** Un par de +5 y -5 masas solares suma cero, y ahí eso dejaba la velocidad de TODOS los cuerpos en NaN nada más cargar, sin un solo error en consola. Ahora, con masa total nula, no se corrige nada: el momento por unidad de masa no significa nada en ese caso.
- **Y un cuerpo de tipo `bh` no puede tener masa negativa**, porque el motor le fija el radio al de Schwarzschild y ese sale negativo. La materia exótica se dibuja como un cuerpo normal.
- `scn.span` y `scn.rscale` **pueden no venir.** Con la curvatura automática el alcance lo decide la calibración, así que los escenarios ya no los declaran. Pero `sim.refSep = scn.span * 0.6` daba NaN, y ese NaN entraba en `inspiral()` y de ahí a las posiciones: las seis escenas de espiral gravitatoria se quedaban vacías. Regla: **todo lo que se lea de `scn` y se meta en un cálculo necesita un valor por defecto.** `refSep` se fija ahora después de calibrar, con el alcance real.
- **Un disco muy pegado a un agujero negro se deshace solo.** El de acreción empezaba a diecisiete radios de Schwarzschild, donde el periodo es de un minuto: basta un paso de integración un poco largo para que una partícula gane energía de la nada y salga disparada. Con el borde interior a 700 radios aguanta y se sigue viendo la rotación diferencial, que es lo que se quería enseñar.
- **La lámina 2D tenía el número de líneas fijo en 56** y no miraba el mando de densidad. Se movía el control de 8 a 60 y en pantalla no cambiaba nada, porque en modo lámina lo que se ve es la lámina y no

la retícula 3D (que sí respondía). Si hay dos representaciones del mismo campo, las dos tienen que leer el mismo mando.

- `primarioDe` (**quien más tira**) **no es la jerarquía orbital**. Ver §5 quater.
- `encuadrar()` **estaba definida dos veces**, idéntica, y ganaba la segunda. Inofensivo hasta que se toca una de las dos.
- **El radio de Schwarzschild son $2,95e-6$ UL por masa solar**. Un factor mil de más en los agujeros negros estelares metía los dos cuerpos de una binaria uno dentro del otro y sacaba NaN.
- **El bloqueo de puntero no puede ser la única forma de girar la cámara**. El navegador lo puede denegar, y al soltarlo el usuario se queda sin controles. Arrastrar con los botones del ratón funciona siempre; la captura es un extra.

8. Prueba de humo

```
python dev/humo.py
```

Abre la aplicación con Chromium headless, recorre escenarios, cambia entre 2D y 3D, lanza rayos y sondas, y **falla si hay un solo error de consola o de shader**. Es la red de seguridad: un shader que no compila no da error visible en pantalla, solo deja de dibujar.

Requiere `playwright` (ya instalado en este equipo).

9. Subida a un subdominio

9.1 Qué subdominio

El elegido y **ya escrito en todos los ficheros** es:

```
orbitas.gmartos.es
```

Razones, por orden de peso:

1. **Una palabra, castellana y sin tilde**. `órbita` con tilde no se puede poner en un dominio; `orbita` se escribe igual que se dice y no hay que deletrearlo por teléfono ni en una pizarra de clase.
2. **Describe el contenido en la propia URL**. Google usa el subdominio como señal débil, pero un profesor que ve `orbitas.gmartos.es` en una diapositiva ya sabe qué va a abrir.
3. **No se pisa con nada**. No colisiona con ningún otro proyecto del dominio.

Alternativas razonables, si se prefiere otra: `universo.gmartos.es` (más amplio, por si luego crece a otras simulaciones), `gravedad.gmartos.es` (lo más literal de lo que enseña) o `orbituniverse.gmartos.es` (marca, pero más largo y en inglés).

Si se cambia, hay que cambiarlo en CINCO sitios o el SEO queda apuntando a una URL que no existe:

Fichero	Qué hay dentro
index.html	<link rel=canonical>, og:url, og:image, twitter:image y los cuatro @id del JSON-LD
guia.html	<link rel=canonical> y los enlaces internos
robots.txt	la línea Sitemap:
sitemap.xml	los dos <loc>
manifest.webmanifest	nada: start_url y scope son relativos (./) a propósito, para que funcione en cualquier dominio

Un `grep -rn "orbitas.gmartos.es" .` los encuentra todos.

9.2 Qué se sube

La carpeta entera **menos** dev/, capturas/ y _bk_*. Es HTML + JS + CSS estáticos: **no hace falta Node en el servidor**, server.js es solo para desarrollar en local.

```
index.html  guia.html  robots.txt  sitemap.xml  manifest.webmanifest
og.png     icono-180.png  icono-512.png
css/       js/         vendor/
```

9.3 Requisitos del servidor

1. **Content-Type: text/javascript en los .js**. Son módulos ES; si el servidor los manda como text/plain el navegador los rechaza y la página se queda en «Generando el espacio-tiempo...» para siempre. En Apache basta con que el mime.types esté al día; si no: `apache AddType text/javascript .js AddType application/manifest+json .webmanifest`
2. **HTTPS**. No por capricho: el canonical y el JSON-LD declaran https, y sin certificado Google trata el sitio como duplicado o directamente lo degrada.
3. **Compresión**. vendor/three/build/three.module.js es ~1,2 MB en claro y ~300 KB con gzip. Con mod_deflate o mod_brotli: `apache AddOutputFilterByType DEFLATE text/html text/css text/javascript application/javascript image/svg+xml`
4. **Caché larga para vendor/**, corta para el resto. vendor/ no cambia nunca; js/ sí, así que una semana es un equilibrio razonable.
5. **Nada de Cross-Origin-Embedder-Policy restrictivo**. No hace falta (no se usan SharedArrayBuffer) y romper el WebGL con una cabecera de seguridad copiada de otro sitio es el fallo más tonto posible.

9.4 Comprobación después de subir

```
curl -sI https://orbitas.gmartos.es/js/main.js | grep -i content-type
curl -sI https://orbitas.gmartos.es/robots.txt | head -1
curl -sI https://orbitas.gmartos.es/sitemap.xml | head -1
curl -sI https://orbitas.gmartos.es/og.png | head -1
```

Los cuatro tienen que dar `200`, y el primero `text/javascript`. Después, en **Search Console**: dar de alta el subdominio, mandar el `sitemap.xml` y pasar la portada por la **prueba de resultados enriquecidos** — debe reconocer `WebApplication`, `LearningResource` y `FAQPage`.

10. El aula (`js/data/lecciones.js` + `js/ui/aula.js`)

Noventa y nueve lecciones repartidas en tres niveles (primaria, secundaria y bachillerato), con 352 pasos y 104 preguntas, que recorren 227 de los 491 escenarios. Cada lección son varios pasos, y cada paso lleva un escenario, un texto, un «qué hacer» y a veces una pregunta de tres opciones.

Dos decisiones que valen más que el código:

1. **La barra del aula vive FUERA del panel.** Si estuviera dentro, en un teléfono habría que abrir el panel para pasar de paso, y el panel tapa justo lo que hay que mirar. Estando fuera, el alumno ve el cielo y la instrucción a la vez.
2. **Toda pregunta se responde COMPROBÁNDOLA en el simulador, no de memoria.** Es la regla que se siguió al escribirlas: si una pregunta se puede contestar sin tocar nada, no vale y se reescribe. Al fallar no se dice «mal»: el texto de `porque` señala el botón, la ficha o el escenario donde se ve.

Un paso puede además ejecutar una acción con `paso.accion`, que se pasa tal cual a `app.whatIf()`. Así una lección puede quitar el Sol o mover la Tierra sin que el alumno tenga que buscar el botón.

Añadir una lección es tocar **un solo fichero**, `js/data/lecciones.js`. Copiar la primera lección del fichero y cambiarle el contenido es lo más rápido; los campos de cada lección son estos (los nombres van sin tilde porque son identificadores de JavaScript, no texto para leer):

Campo de la lección	Qué lleva
<code>id</code>	clave única en minúscula con guiones
<code>nivel</code>	<code>Primaria</code> , <code>Secundaria</code> o <code>Bachillerato</code> ; agrupa el índice
<code>titulo</code>	el nombre que se ve en el índice y en la cabecera
<code>duracion</code>	texto libre, del tipo «10 min»
<code>resumen</code>	una frase bajo el nombre, en el índice
<code>pasos</code>	la lista de pasos

Y los de cada paso:

Campo del paso	Qué lleva
<code>escenario</code>	clave del catálogo; se carga al entrar en el paso
<code>titulo</code>	se ve junto a «Paso 3 de 7»
<code>texto</code>	la explicación
<code>mirar</code>	la instrucción concreta, precedida de «Qué hacer:»
<code>accion</code>	opcional; se pasa tal cual a <code>app.whatIf()</code>
<code>pregunta</code>	opcional; enunciado, tres opciones, cuál es la buena y el porqué

El `escenario` tiene que existir en el catálogo: si no, el paso sigue siendo legible pero no carga nada (está envuelto en `try`), y eso es un fallo silencioso. `python dev/cuenta.py` no lo caza; se ve recorriendo la lección.

11. Móvil: qué se reorganizó y por qué

El diseño de escritorio no se «encoge»: en un teléfono hay cosas que cambian de sitio, y otras que desaparecen.

Elemento	Escritorio	Teléfono (≤ 820 px)
Panel	columna fija de 326 px a la izquierda	hoja que sube desde abajo, 74 vh, con asa
Panel al arrancar	abierto	plegado , y en modo simple
Mando de cámara	esquina inferior derecha, siempre	escondido , con su propio botón redondo
HUD	abajo a la izquierda, completo	arriba a la derecha, cuatro filas (tres por debajo de 480 px)
Leyenda de color	visible	oculta

Tres fallos medidos y corregidos, que son la razón de que esto esté escrito:

- **La ficha se iba media pantalla fuera.** En escritorio se centra con `left:50% + translate(-50%,...)`. Al pasar a `left/right` en el móvil hay que **anular el `transform`**, o la tarjeta ocupa de -175 a 195 px en una pantalla de 390. Ahora `transform:translate(0, -14px)`.
- **El mando tapaba el buscador.** Con la hoja abierta, el mando se pintaba de x=253 a x=390, encima del panel, y cortaba por la mitad el campo de búsqueda. Se esconde en móvil (`html.pad-visible` para sacarlo) y **nunca** se muestra con el panel abierto. No se pierde nada: se rota arrastrando y se acerca pellizcando.

- **Objetivos de toque de 15 px.** Casillas, deslizadores, botones del mando y la X de cerrar estaban entre 15 y 30 px. Todos a 40-44.

Lo comprueba `python dev/movil.py`, que recorre cuatro tamaños (375, 390, 412 y 820 px), guarda 28 capturas y mide las tres cosas que de verdad rompen una interfaz móvil: **desborde** (algo más ancho que la ventana), **objetivo de toque pequeño** y **el panel tapando su propio botón de cerrar**.

Un detalle del medidor: lo que se mide es **la zona que se pulsa**, no el elemento. Una casilla de 22 px dentro de una etiqueta de 44 se acierta perfectamente, porque el dedo cae en la etiqueta. Medir el `<input>` a secas daba 16 falsos positivos.

12. SEO: qué se añadió y para qué sirve cada cosa

Una aplicación WebGL es, para un rastreador, una página en blanco: todo el contenido lo pinta JavaScript en un `<canvas>`, y un `<canvas>` no tiene texto. Sin nada más, Google indexa un título y punto. Lo que se ha puesto:

Pieza	Para qué
<code><title></code> de 72 caracteres y <code>description</code> de 205	lo que se ve en el resultado de búsqueda
<code><link rel=canonical></code>	evita que <code>www</code> , <code>/index.html</code> y <code>/</code> cuenten como tres páginas
Open Graph + Twitter card + <code>og.png</code> 1200×630	la tarjeta al compartir el enlace en WhatsApp, Slack o X
JSON-LD <code>WebApplication</code>	ficha de aplicación: categoría, precio 0, requisitos
JSON-LD <code>LearningResource</code>	la pieza clave para el uso escolar: declara nivel educativo y qué enseña
JSON-LD <code>FAQPage</code>	cuatro preguntas que pueden salir desplegadas en el resultado
<code><noscript></code> con el contenido real	texto indexable de verdad: qué es, las ocho familias de escenarios, el aula y el motor
<code>guia.html</code> (1.112 palabras)	una página de texto que sí se puede posicionar, con enlaces a la aplicación
<code>robots.txt</code> + <code>sitemap.xml</code>	rastreo

Un fallo que estuvo dentro y se quitó: el `robots.txt` llevaba `Disallow: /vendor/`, porque parecía una dependencia interna que no aporta nada al buscador. Es justo al revés. Google renderiza la página con un Chrome sin ventana; si no puede bajar `three.module.js`, lo que ve es un `<canvas>` vacío y trata el sitio como una página rota. **Nunca se bloquea el JavaScript ni el CSS que la página necesita para dibujarse.** Solo `capturas/` sigue bloqueado, que son megas de PNG de desarrollo. | `manifest.webmanifest` + iconos | instalable en el móvil como aplicación |

El `<noscript>` **no es texto escondido**: es lo mismo que cuenta la aplicación, y quien llegue sin WebGL necesita saber qué hay dentro. Esa es la diferencia entre contenido alternativo legítimo y relleno para el buscador.

13. El impacto gigante: cuatro fallos que no daban ningún error

El escenario `theia` no formaba ninguna Luna. El barrido de los 491 daba **0 errores de consola y 0 NaN**, y aun así el resultado físico era falso: los escombros acababan estirados en un anillo alrededor del **Sol**, no de la Tierra. Lo encontró el usuario mirando la pantalla.

Eran cuatro fallos encadenados, y los cuatro empeoraban justo al acelerar, que es lo primero que hace cualquiera con un escenario de choque.

13.1 La eyecta nació en una posición caducada

El evento de fusión guarda una copia (`ev.centro`, `ev.vBase`, `ev.punto`) de dónde estaba el planeta **en el instante del choque**. Pero las partículas se creaban después: la primera oleada un fotograma más tarde, y la segunda con un `setTimeout` de 700 ms de **reloj de pared**, que no tiene ninguna relación con el tiempo del escenario.

A 200.000 días por segundo, un fotograma son 3,3 días, y la Tierra recorre 10 UL: **mil seiscientos radios terrestres**. Medido antes de arreglarlo: el 82 % de los trozos aparecía entre 100 y 1.000 radios del planeta, y en la franja donde tiene que estar el disco (1,7 a 4,7 radios) había **una** partícula de 1.279.

Arreglo: `_emitirOleada()` lee la posición y la velocidad del **cuerpo vivo** (`ev.cuerpo`), no la copia del evento, y la segunda oleada se encola por **tiempo simulado** (`_oleadas`, `_tickOleadas()`), seis horas de simulación después del contacto.

13.2 Los escombros iban `sub` veces más despacio que los cuerpos

```
this.sim.step(days, sub)           // la Tierra avanza 3,3 días
this.debris.step(days / sub, ...)   // su eyecta avanza 0,014
```

Los cuerpos avanzaban `days` enteros en `sub` subpasos; los escombros, **un solo subpaso**. Con 240 subpasos, el planeta se marchaba y dejaba su propio disco atrás. Y cuanto más se aceleraba, mayor era `sub` y peor el desfase.

Arreglo: `Debris.step(dt, bodies, radiusOf, limite, nSub)` da `nSub` subpasos (topados en 48 para que el coste no se dispare), con el mismo tiempo que los cuerpos.

13.3 El anfitrión del disco era el Sol

`formarLuna()` y `condensarCumulo()` elegían el anfitrión así:

```
const host = this.bodies.filter(...).sort((a, b) => b.mu - a.mu)[0];
```

El cuerpo más masivo de la escena. En el impacto de Theia, **el Sol**. Con el Sol de anfitrión, el umbral `dd.masa >= host.mass * 0.004` pedía escombros por encima del 0,4 % de una masa solar —unas **mil trescientas Tierras**— y el radio de la órbita resultante ($a = |h|^2/\mu$) salía con μ del Sol, o sea 1 UA. **No se formaba una luna en ningún escenario que tuviera una estrella dentro.**

Arreglo: `anfitriónEscombros()`. Y el primer intento también estaba mal: contar la masa ligada a cada cuerpo **volvía a dar el Sol**, porque un trozo que orbita a la Tierra orbita también al Sol. Lo correcto es asignar cada partícula al anfitrión **más local**: el de esfera de Hill más pequeña que la contenga y al que esté ligada. Y `discoExterior()` solo cuenta material ligado y dentro de la esfera de Hill: antes entraban las diez masas lunares que se marchan para no volver, cuyo momento angular respecto de la Tierra apunta a cualquier lado.

13.4 Y cuando por fin nacía, la Tierra se la comía

`collideScale: 700` le da a la Tierra un radio de colisión de **700 radios terrestres**. La Luna se formaba correctamente a 3-5 radios, con su masa y su momento angular bien calculados, y desaparecía en el mismo fotograma. `formarLuna()` devolvía `true` y no quedaba ninguna luna.

Arreglo, en `collideRadius()`: el radio de colisión se topa al **45 % de la distancia al satélite propio más cercano, y también al 45 % de la distancia a su anfitrión**. Las dos direcciones, porque la Luna recién formada mide 0,25 radios terrestres y multiplicada por 700 son 176: era **ella** la que se comía a la Tierra. Durante el impacto el tope no estorba, porque el impactador todavía no orbita a nadie.

13.5 El resultado, contrastado

`python dev/theia.py` mide lo que de verdad decide si nace una luna:

```
impacto: eyecta 11,5 masas lunares (0,141 M_T) · v_esc 11,06 km/s · radio 6.297 km
día 24: 1,54 masas lunares ligadas, todas fuera del límite de Roche, a 2,8 R_T
después: una Luna de 1,06 masas lunares, e = 0,01, a 26,3 radios terrestres,
         estable durante los 105 días medidos
```

Las simulaciones hidrodinámicas publicadas del impacto canónico (Canup 2004, Čuk & Stewart 2012) dan un disco de 1,5 a 2 masas lunares y una luna de ~1 masa lunar a 3-5 radios terrestres. Cuadra.

13.6 La lección

Un barrido sin errores no prueba nada sobre el resultado. Lo que caza esto es una prueba que mida **el desenlace esperado** y lo compare con la literatura. Son cuarenta líneas (`dev/theia.py`) y se escribió después, no antes.

Lo mismo pasó con otras dos cosas el mismo día:

- `dev/velocidad.py` encontró **23 escenarios que avanzaban más días por fotograma de los que dura su órbita más corta**. `comp-m87` declaraba 200.000 días por segundo con estrellas de 149 días de periodo: veintidós vueltas enteras entre un fotograma y el siguiente. No es un error, es una mentira. Se corrige en `frenaSiVuela()`, que al cargar topa la velocidad a cinco fotogramas por órbita —solo hacia abajo, y sin tocar el deslizador.
- `dev/aula.py` encontró una lección que apuntaba a un escenario **inexistente**. No da error: el paso simplemente no carga nada.

14. Viaje en el tiempo: no perder de vista lo que se está mirando

Un salto de diez millones de años mueve todo: `_anclaBaricentro()` vuelve a poner el baricentro en el origen y cada cuerpo aparece en otro punto de su órbita. La cámara se quedaba donde estaba y el sistema se salía del cuadro; en modo nave no hay seguimiento que lo arregle solo.

`_anclaVista()` apunta, **antes** de saltar, el desplazamiento de la cámara respecto del cuerpo enfocado (el objetivo, o el más masivo si no hay). `_restauraVista()` lo vuelve a aplicar sobre su posición nueva y rehace `orbitDist`, `orbitPhi` y `orbitTheta` del vector, poniendo `orbitPan` a cero. Misma distancia, mismo ángulo, mismo cuerpo en el centro; lo que cambia es lo que hay alrededor, que es justo lo que se ha ido a ver.

Se pasa a `_trasSaltar(ancla)` por los dos caminos: el salto integrado y el propagado con Kepler.

15. El paso de integración es FIJO, y por qué eso importa tanto

Informe del usuario: «le doy velocidad a un sistema y un planeta que debería seguir ahí miles de millones de años se sale de órbita».

No era física. Velocity-Verlet es **simpléctico**: con paso constante, el error de energía se queda acotado y oscila, no crece. Esa propiedad **se pierde en cuanto el paso cambia**, y aquí cambiaba en cada fotograma, porque el avance salía del tiempo real transcurrido:

```
days = speed * dt          <- dt es distinto en cada fotograma
sub   = ceil(days / maxStep)
paso  = days / sub         <- distinto en cada fotograma
```

Con paso variable la deriva deja de estar acotada y pasa a crecer sin freno. Un cuerpo que gana energía sin parar acaba escapando.

Medido con `dev/pasofijo.py`, sistema solar, 3.000 años simulados y el mismo paso medio en los dos casos:

paso	mayor cambio de semieje
variable	88,8 % (Ío)
fijo	0,5 % (la Luna)

La corrección

La integración avanza en pasos de **exactamente** `maxStep`, y el sobrante de cada fotograma se guarda para el siguiente:

```

this._deuda += days;
const h = this.maxStep;
let n = Math.trunc(this._deuda / h);
if (Math.abs(n) > tope) n = Math.sign(n) * tope;
this._deuda -= n * h;
if (n !== 0) this.sim.step(h * n, Math.abs(n), ...); // cada subpaso es h EXACTO

```

Si en un fotograma no cabe ni un paso entero (velocidad muy baja), no se avanza: se acumula. Lo que **no** se puede hacer es dar un pasito distinto, que es justo lo que rompía la simetría. Se sigue dibujando, claro.

La comprobación

`python dev/accelera.py` deja correr **la aplicación de verdad**, con su bucle y la velocidad al máximo, sobre catorce escenarios que llevan ahí miles de millones de años, y comprueba que los semiejes siguen donde estaban:

escenario	años	dE/E	escapados	mayor cambio de semieje
sistema	24	3,9e-06	0	1,675 % (Luna)
sistema-completo	1793	6,9e-07	0	0,734 % (Plutón)
sol-tierra	3074	8,5e-07	0	1,619 % (Luna)
res-2-1	35219	1,2e-04	0	0,114 % (Exterior)
lagrange	10883	1,3e-06	0	0,000 % (18 testigos aparte)
alfa-centauri	37318	2,1e-08	0	0,003 %
cinturon	250	2,0e-06	0	0,065 % (90 testigos aparte)

Las partículas de prueba se miden aparte, y no es una excusa. Un troyano en L4/L5 o un asteroide junto a una resonancia **libra**: su semieje oscila varios por ciento y vuelve, y eso es exactamente lo que el escenario quiere enseñar. Medirles «deriva» no significa nada. Lo que sí tiene que quedarse quieto es un cuerpo con masa de verdad, y se queda: el peor es la Luna con un 1,7 %, que es la perturbación solar auténtica.

El avance rápido no toca esto

`opts.turbo` sube el número de subpasos por fotograma según lo que la máquina pueda calcular en 22 ms, medido sobre el coste real del fotograma anterior. El **tamaño** del paso no cambia: sigue siendo `maxStep`. Por eso el resultado es idéntico a haberlo visto pasar despacio, solo que mucho antes.

16. Transferencia de masa por lóbulo de Roche

La lección decía «sigue la masa de la enana blanca mientras recibe material» y no había ninguna transferencia: el escenario eran dos enanas blancas fusionándose, que es otro canal distinto. Se contaba algo que no ocurría.

El modelo

Alrededor de cada estrella de un binario cerrado hay una región, el lóbulo de Roche, más allá de la cual el material ya no le pertenece. El radio del lóbulo es la fórmula de **Eggleton (1983)**, que aproxima el resultado

exacto con un error menor del 1 % para cualquier razón de masas:

$$r_L / A = 0,49 \, q^{(2/3)} / (0,6 \, q^{(2/3)} + \ln(1 + q^{(1/3)})), \quad q = M_{\text{donante}} / M_{\text{receptor}}$$

Cuando el radio del donante supera su lóbulo, cede masa por el punto L1. El **ritmo** no pretende ser el real (uno real tarda millones de años): se escala con `scn.transferencia` para que se vea. Lo que sí es real es quién da y quién recibe, cuándo empieza, y que la masa y el momento se conservan.

Las consecuencias, que es lo que se ve

1. **Los radios se rehacen, cada uno con su ley.** Una estrella de secuencia principal y un objeto compacto siguen `classify()`; una estrella **evolucionada** (una gigante, una subgigante) no, porque su radio es el de una envoltura hinchada. Al perder masa se hincha **más**: $R \propto M^{-1/3}$ para una envoltura convectiva. Eso es lo que hace que la transferencia **se realmente** en vez de pararse sola.
2. **Una enana blanca que engorda ENCOGE**, porque su relación masa-radio es la inversa. Se ve en la ficha mientras pasa.
3. **Al cruzar el límite de Chandrasekhar (1,44 M $\odot$), detona.** Es el canal simple de las supernovas de tipo Ia. No deja resto: toda la masa sale despedida a 10.000 km/s.

Medido

`python dev/supernova.py`, escenario `supernova-ia-acrecion`:

día	enana M $\odot$	enana R $\odot$	compañera M $\odot$	total M $\odot$
0	1,1000	0,00812	1,2000	2,3000
65	1,2609	0,00782	1,0391	2,3000
81	1,3877	0,00758	0,9123	2,3000
86	DETONA • 900 fragmentos con 1,4403 M $\odot$			2,2995

Masa conservada al **0,02 %**. Y la enana ha encogido un 7 % mientras engordaba un 31 %.

Cinco fallos que costaron, y ninguno daba error

- **El radio se recalculaba ANTES de comprobar Chandrasekhar.** `classify(m, colapsado)` con m por encima de 1,44 devuelve una estrella de **neutrones**, así que la comprobación `kind == 'wd'` ya no se cumplía nunca. La enana llegaba a 2,3 M $\odot$ sin explotar.
- **La compañera hinchada se convertía en secuencia principal** en la primera transferencia: de 3 radios solares a 1,1, dejaba de llenar su lóbulo, y todo se quedaba clavado para siempre.
- **La separación era menor que el radio de la compañera** (2,2 contra 3 radios solares): se fusionaban en el fotograma uno. Con la fórmula de Eggleton, la separación tiene que estar entre el contacto (3,01) y el desbordamiento (7,8).
- **La velocidad de expulsión, mal dos veces seguidas.** Primero `vEsc × 3` de la propia enana blanca, que son 21.700 km/s (el doble de lo que toca). Y después, «corrigiéndolo», 0,86 UL/día, que son **10 km/s**: mil veces menos, y la eyecta volvía a caer entera. **1 UL/día son 11,57 km/s**, así que 10.000 km/s son 864.
- **El detector de cúmulos recondensaba la eyecta de la supernova** en un cuerpo de 0,43 masas solares al instante siguiente de la explosión. La comprobación de energía no bastaba: justo al detonar los

fragmentos están todos dentro del radio del cuerpo que ha reventado, y con un radio tan pequeño la energía de ligadura sale enorme. Ahora se mira además la velocidad **radial** media: un grupo que se aleja de su propio centro no es un cúmulo, es una explosión.

Y la lección de todo esto

Ninguno de los cinco producía un error de consola, ni un NaN, ni un aviso en el barrido. Los caza una prueba que **mide el desenlace esperado** y lo compara con lo que dice la literatura. Eso es `dev/supernova.py`, y son ochenta líneas.

Orbit Universe — el modelo físico

Qué es exacto, qué es una aproximación conocida y qué está exagerado a propósito. Escrito para poder defender cada píxel de la pantalla.

1. El problema de las láminas 2D

La imagen de referencia —la sábana hundida con una bola encima— tiene tres cosas mal, y una bien:

Mal

1. **Sobra una dimensión y falta otra.** El espacio tiene tres dimensiones; el dibujo pinta dos y usa la tercera para representar la curvatura. En el universo real esa tercera dirección **no existe**: no hay un "abajo" al que caer.
2. **Explica la gravedad con la gravedad.** La bola rueda hacia el centro porque hay una gravedad *fuera* del dibujo que tira de ella hacia abajo. Es circular.
3. **Es espacio, no espacio-tiempo.** La lámina solo dibuja la geometría del espacio. La parte que de verdad produce casi toda la gravedad newtoniana es la curvatura del **tiempo**: dos relojes a distinta altura no marcan lo mismo, y esa es la razón de que una manzana caiga.

Bien

4. La **forma** del hundimiento sí es correcta. Es el paraboloides de Flamm.

Este simulador arregla el 1 y el 2 (la retícula se contrae en volumen, sin ninguna dirección extra y sin ninguna gravedad externa), conserva el 4 en el modo lámina, y del 3 enseña la parte visible: el HUD muestra la **dilatación temporal** en cada punto por el que vuelas.

2. Unidades

UL = 10^9 m	día = 86.400 s	masa = kg
GU = $4,98213 \cdot 10^{-28}$	$\text{UL}^3/(\text{kg} \cdot \text{día}^2)$	
c = 25.902,068	UL/día	
rs = $2GM/c^2$	$\rightarrow 1,4853 \cdot 10^{-36} \cdot M[\text{kg}]$	UL

Comprobación: $\mu_{\odot} = \text{GU} \cdot M_{\odot} = 990,9 \text{ UL}^3/\text{día}^2$. A $r = 149,598 \text{ UL}$, $v = \sqrt{\mu/r} = 2,574 \text{ UL/día}$ y $T = 2\pi r/v = 365,2 \text{ días}$. Sale solo.

3. La curvatura que se dibuja

3.1 Modo 3D — contracción radial

$$r' = r \cdot (r_f / (r_f + R_v))^w \quad r_f = \max(r, R_{\text{cuerpo}})$$

Justificación: en la métrica de Schwarzschild en coordenadas isotópicas, cerca de una masa **cabe más distancia propia de la que cabría en plano**. Dibujar una retícula de coordenadas con esa distancia propia significa apretujar las celdas hacia el cuerpo. Es el análogo tridimensional exacto del hundimiento de la lámina:

- lejos, $1 - r'/r \approx w \cdot R_v/r$ → **cae como el potencial**, igual que la lámina;
- cerca, $r' \rightarrow 0$ → la garganta;
- r_f acotado por el radio del cuerpo: dentro de una masa la geometría se aplanan, como en la solución interior de Schwarzschild.

No es la métrica resuelta numéricamente: es una **familia de curvas con el comportamiento asintótico correcto** y un parámetro de exageración. Se dice claramente en pantalla.

3.2 Modo 2D — paraboloide de Flamm (exacto)

$$z(r) = 2 \cdot \sqrt{(r_s \cdot (r - r_s))}$$

Ésta sí es exacta: es la incrustación isométrica de una rebanada $t = \text{cte}$, $\theta = \pi/2$ de Schwarzschild en un espacio euclídeo de 3 dimensiones. Se dibuja invertida ($z_{\text{max}} - z(r)$) para que el embudo baje, que es como se ve en todas partes.

3.3 Lo que decide la diferencia entre Sol, enana blanca, estrella de neutrones y agujero negro

La misma regla para los cuatro. Lo único que cambia es el suelo r_f : hasta dónde te puedes acercar antes de tropezarte con la superficie.

Objeto	R / r_s
Sol	235.500
Enana blanca (Sirio B)	~3.900
Estrella de neutrones (1,4 $M_{\odot}$)	~2,7
Agujero negro	1 (no hay superficie)

En el escenario **Comparativa de pozos** el suelo respeta esa relación real, normalizada para que el Sol quepa en pantalla. El resultado es el dibujo clásico del Sol / estrella de neutrones / agujero negro, pero con números detrás.

3.4 La exageración

$R_v = \text{warp} \cdot (\mu/\mu_\odot)^\gamma$ es un **radio de Schwarzschild visual**. El real del Sol son **2,95 km** = $2,95 \cdot 10^{-6}$ UL, frente a una escena de cientos de UL: sin exagerar no se vería absolutamente nada. El HUD muestra siempre el valor real al lado del visual.

$\gamma < 1$ realza los cuerpos pequeños. Con $\gamma = 1$ (proporción real) la Tierra tiene un pozo 333.000 veces menos profundo que el Sol y desaparece.

4. Órbitas — sin exagerar nada

Newton puro, con masas y distancias reales, integrado con velocity-Verlet (simplicítico: la energía no deriva). Suavizado de Plummer en el radio del cuerpo. El paso máximo sale de la órbita más corta del sistema dividida entre 80.

La precesión relativista del perihelio se añade aparte, como corrección de primer orden opcional (§4 bis). Está apagada por defecto: a escala real son 43" por siglo, o sea nada en una simulación de unos miles de días.

4 bis. Precesión relativista del perihelio (1PN)

Con Newton la elipse se cierra. Con la relatividad general no: el perihelio avanza. La corrección de primer orden para una partícula alrededor de una masa dominante es

$$a = -\mu r/r^3 + (\mu / (c^2 r^3)) [(4\mu/r - v^2) r + 4 (r \cdot v) v]$$

y da un avance de $6\pi\mu / (c^2 a (1-e^2))$ por vuelta. Para Mercurio:

$$6\pi \cdot 1,327e20 / (8,988e16 \cdot 5,79e10 \cdot (1 - 0,2056^2)) = 5,02 \cdot 10^{-7} \text{ rad/órbita} \\ \rightarrow 415 \text{ órbitas por siglo} \rightarrow 2,08 \cdot 10^{-4} \text{ rad} = 42,98 \text{ segundos de arco por siglo}$$

que es exactamente el residuo que Le Verrier no supo explicar en 1859 y que Einstein calculó en noviembre de 1915. **Es real y está implementado tal cual**, pero $5 \cdot 10^{-7}$ radianes por órbita no se ven en una simulación de unos miles de días: de ahí el factor de exageración del panel. Con $\times 1.000.000$ el perihelio gira unos 29° por vuelta y se ve de inmediato. Apagándolo ($\times 1$) la elipse vuelve a cerrarse.

No se incluyen el arrastre de marcos (Lense-Thirring) ni términos de orden superior.

4 ter. Geometría del campo

Puntos de Lagrange, esfera de Hill, límite de Roche y zona habitable son **exactos** (o las aproximaciones estándar), resueltos en vivo. Ver [DOC-TECNICO.md](#) §5 bis.

La **curva de rotación** compara la velocidad circular calculada con toda la masa encerrada y con solo la visible. Para el halo se usa la masa encerrada de Plummer, $M r^3 / (r^2 + b^2)^{3/2}$. Los puntos son los cuerpos reales de la escena: caen sobre la curva de la masa total, no sobre la de la materia visible. Ésa es la observación, hecha en miles de galaxias, que obligó a postular la materia oscura.

5. Luz

En campo débil, la desviación de un rayo que pasa a parámetro de impacto b es

$$\alpha = 4GM / (c^2 b)$$

el doble que la newtoniana. Por eso los fotones se integran con $a = 2 \cdot a_{\text{Newton}}$ y se renormaliza $|v| = c$ en cada paso: reproduce exactamente la desviación total a primer orden.

Los rayos usan las masas **visuales** ($\mu_{\text{vis}} = Rv \cdot c^2 / 2$), no las reales, para que lo que ves en la malla y lo que hace la luz sean coherentes. Con las reales la desviación por el Sol son **1,75 segundos de arco**: cero píxeles.

Campo fuerte: el agujero negro

Ahí no vale la aproximación débil. Se integra la geodésica nula completa:

$$d^2 u / d\phi^2 + u = 3Mu^2 \quad \Leftrightarrow \quad a = -1,5 \cdot h^2 \cdot p / |p|^5 \quad (r_s = 1)$$

con $h = p \times d$ constante. Eso da, sin ponerlos a mano:

- la **esfera de fotones** en $r = 1,5 r_s$ y la sombra de radio $\approx 2,6 r_s$;
- el **anillo de fotones** en el borde;
- el **arco del disco por encima del agujero**: la cara de atrás del disco vista por arriba, porque un mismo rayo cruza el plano del disco varias veces;
- el **arrastré** de las estrellas del fondo.

Sobre el disco se aplican: - **realce Doppler relativista** $(1 - \beta)^{-3}$ con $\beta = v \cdot \hat{n}$ y $v_{\text{orb}} = \sqrt{M/r}$ limitada a $0,72 c$ — de ahí que un lado brille mucho más que el otro; - **corrimiento al rojo gravitatorio** $\sqrt{1 - r_s/r}$; - rotación **diferencial kepleriana**: el borde interno gira mucho más rápido.

No se modela el arrastre de Kerr (agujeros negros en rotación): todo es Schwarzschild.

6. Dilatación temporal

El HUD muestra $\sqrt{1 - 2\Phi/c^2}$ con $\Phi = \sum \mu_i / r_i$ (aproximación de campo débil). Es lo que marca el reloj de la nave frente a un reloj en el infinito. En el sistema solar sale 1,000000 salvo si te pegas al Sol; junto a un agujero negro se desploma.

Ésta es la parte del espacio-**tiempo** que las láminas 2D no cuentan, y es la que produce casi toda la gravedad que sentimos.

7. Colisiones: nube de partículas, no una fusión instantánea

Fundir dos cuerpos en uno de golpe es una mentira cómoda. Ahora el resultado depende de lo violento que sea el choque, siguiendo el modelo de **Leinhardt y Stewart** simplificado:

$$Q_R = (1/2) \mu_r v^2 / M \quad \text{energía específica del impacto}$$

$$Q_D \approx (3/5) G M / R \quad \text{energía específica de ligadura del cuerpo resultante}$$

$$Q_R < 0,08 Q_D \rightarrow \text{acreción limpia: se funden}$$

$$0,08 - 1,8 Q_D \rightarrow \text{impacto erosivo: } M_{LR}/M = -0,5 (Q_R/Q_D - 1) + 0,5$$

$$> 1,8 Q_D \rightarrow \text{catastrófico: casi todo se pulveriza}$$

Lo que no queda en el remanente sale como **partículas con masa** que sienten: - la gravedad de los cuerpos grandes, - la gravedad **de las demás partículas**, con una malla gruesa (*particle-mesh*), - un suavizado del tamaño de celda que hace de presión.

Las que vuelven a caer se **acretan** (masa y momento se conservan); las que se van más allá de doce veces el alcance se descartan. Si un montón acaba girando junto y ligado entre sí, deja de ser polvo y se convierte en un **cuerpo nuevo**.

Lo que está impuesto y no resuelto: una fracción de la eyecta (por defecto la mitad) se coloca directamente en órbita entre 1,7 y 4,7 radios. En una simulación hidrodinámica eso lo produce la presión del choque, que aquí no se resuelve. Sin ese disco no hay de dónde salga una luna, porque **todo lo que se lanza desde la superficie vuelve a la superficie**. El resto —el reparto de masa, el reparto de velocidades acotado por la velocidad de escape *local*, la reacreción y la autogravedad— sí sale del cálculo.

Medido en el escenario a escala real: de 546 fragmentos, 339 se quedan en un disco estable entre 1,6 y 5 radios terrestres y 287 vuelven a caer; la Tierra pasa de 1,000 a 1,062 masas terrestres. Formar la Luna de ese disco lleva, en la realidad, entre décadas y siglos de evolución del disco: eso no cabe en la simulación.

Detección de choque por **barrido**: distancia mínima del segmento recorrido en el paso. Sin esto, con pasos de horas, dos planetas a 30 km/s se atraviesan sin enterarse.

8. Ondas gravitatorias

Al fusionarse dos cuerpos se emite un pulso esférico: envolvente gaussiana, oscilación senoidal y amplitud que decae como $1/r$. **No es una solución de las ecuaciones de Einstein linealizadas**: es una representación cualitativa con la forma correcta (pulso que se aleja, decae con la distancia, deforma la geometría a su paso). No tiene el carácter cuadrupolar transverso ni las polarizaciones $+$ y $\times$.

La espiral previa a la fusión sí tiene la forma correcta —la separación se reduce y la frecuencia sube: el *chirp*— pero está **acelerada varios órdenes de magnitud**. GW150914 real recorrió sus últimas órbitas en 0,2

segundos, tras miles de millones de años de espiral.

9. Materia oscura

El halo se modela con un **potencial de Plummer**, $\Phi = -GM/\sqrt{(r^2 + b^2)}$, no con un perfil NFW. Basta para lo que enseña: dentro del radio de escala la masa encerrada crece con el radio, así que las órbitas exteriores van mucho más rápido de lo que la materia visible permitiría. Ésa es la observación que obligó a postular la materia oscura.

10. Relaciones masa-radio del laboratorio

Régimen	Relación	Límite
Rocoso	$R \propto M^{0,27}$	hasta $\sim 2,5 M_{\oplus}$
Supertierra / minineptuno	$R \propto M^{0,55}$	hasta $\sim 0,4 M_{\oplus}$
Gigante gaseoso	R casi constante	$0,4 - 13 M_{\oplus}$
Enana marrón	$R \approx 0,85 R_{\oplus}$	$13 M_{\oplus} - 0,075 M_{\odot}$
Enana roja	$R \propto M^{0,80}$	hasta $1 M_{\odot}$
Secuencia principal	$R \propto M^{0,57}$	$1 - 8 M_{\odot}$
Estrella masiva	$R \propto M^{0,55}$	$8 - 150 M_{\odot}$
Enana blanca	$R \propto M^{(-1/3)}$	hasta Chandrasekhar, $1,44 M_{\odot}$
Estrella de neutrones	$R \approx 11-12 \text{ km}$	hasta TOV, $\sim 2,17 M_{\odot}$
Agujero negro	$R = r_s$	sin límite

El caso llamativo es la enana blanca: **a más masa, menos radio**. Es la firma de la presión de degeneración de los electrones, y es lo que hace que exista el límite de Chandrasekhar.

Son ajustes empíricos de uso corriente, no modelos de estructura estelar.

11. Resumen honesto

Concepto	Estado
Masas, radios, distancias, catálogo	Reales
Órbitas y periodos	Reales (Newton + Verlet)
Precesión relativista (1PN)	Exacta a primer orden; exagerada para verla
Puntos de Lagrange, Hill, Roche, zona habitable	Calculados , no puestos a mano
Resonancias de movimiento medio	Detectadas de los periodos reales
Energía y momento angular	Medidos , con su deriva a la vista
Radios de Schwarzschild mostrados	Reales
Forma del pozo (asintótica)	Correcta
Lámina 2D	Exacta (Flamm)
Geodésicas del agujero negro	Exactas (Schwarzschild sin rotación)
Desviación de la luz en campo débil	Exacta a primer orden
Dilatación temporal	Campo débil
Energías de impacto	Reales
Reparto de masa en un choque	Modelo publicado (Leinhardt y Stewart)
Disco de escombros	La mitad impuesta en órbita; el resto, calculado
Relaciones masa-radio	Ajustes empíricos
Profundidad del pozo	Exagerada (se indica el valor real)
Espiral y ondas gravitatorias	Aceleradas y cualitativas
Halo de materia oscura	Plummer, no NFW
Agujeros negros en rotación (Kerr)	No modelados